drones

MDPI

Article

Scalable Pursuit–Evasion Game for Multi-Fixed-Wing UAV Based on Dynamic Target Assignment and Hierarchical Reinforcement Learning

Mulai Tan [1], Haocheng Sun [2], Dali Ding [1,*], Huan Zhou [1] and Yongli Liu [3]

[1] Aviation Engineering School, Air Force Engineering University, Xi'an 710038, China; kgdtml@163.com (M.T.); kgy_zhouh@163.com (H.Z.)
[2] School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China; sunhaocheng@bupt.edu.cn
[3] 93420 Forces, Shijiazhuang 050011, China; 19809748231@163.com
* Correspondence: kgdddl@sina.com; Tel.: +86-137-2075-1846

Highlights

What are the main findings?

- A hierarchical collaborative game framework combining hierarchical reinforcement learning methods and target allocation methods is proposed for autonomous pursuit–evasion.
- The hierarchical reinforcement learning method based on trajectory prediction and stable auxiliary gradients effectively improves the win rate and generates a smooth flight path.

What are the implications of the main findings?

- The simulation of various large-scale pursuit–evasion games demonstrates the advantages of this framework in terms of training time and large-scale scalability.
- The hierarchical reinforcement learning method based on trajectory prediction and stable auxiliary gradients provides a feasible approach for application to real UAVs.

Abstract

The unmanned aerial vehicle (UAV) pursuit–evasion game is the fundamental framework for promoting autonomous decision-making and collaborative control of multi-UAV systems. Faced with the limitations of current deep reinforcement learning methods in terms of transferability and generalization for scalable multi-fixed-wing UAV pursuit–evasion game scenarios, this paper proposes a hierarchical collaborative pursuit–evasion game framework based on target allocation and hierarchical reinforcement learning. The framework comprises three layers: target allocation layer, maneuver decision-making layer, and flight control layer. The target allocation layer employs a dynamic target assignment method based on a dynamic value adjustment mechanism, decomposing the multi-vs.-multi pursuit–evasion game into several one-vs.-one confrontations. The maneuver decision-making layer utilizes a maneuver decision-making method based on trajectory prediction and hierarchical reinforcement learning to generate adversarial maneuver commands. The flight control layer adopts a stable gradient-assisted reinforcement learning flight controller to ensure stable UAV flight. Comparisons with other algorithms across 3V3, 6V6, 9V9, and 12V12 scenarios demonstrate that the proposed method achieves high win rates in diverse game scales. The comparison results also demonstrate the advantages of the framework proposed in this paper in terms of training efficiency and large-scale scalability.

Check for updates

Academic Editor: Pablo Rodríguez-Gonzálvez

Received: 11 November 2025
Revised: 18 December 2025
Accepted: 19 December 2025
Published: 23 December 2025

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Keywords: scalability; target allocation; hierarchical reinforcement learning; pursuit–evasion game; stability-assisted gradient; trajectory prediction

1. Introduction

With the intelligence level improvement of unmanned aerial vehicles (UAVs), they are currently used widely in many fields, such as regional search and rescue [1], relaying communications [2], and disaster relief [3]. Among them, the protection of civil airport airspace is a key issue of people's livelihood security that needs to be urgently addressed at present. Illegal intrusion of UAVs may interfere with flight take-offs and landings and cause safety accidents. Using fixed-wing drones to drive away or capture illegal intrusion UAVs has become a new response measure [4]. UAV pursuit–evasion games provide a basic framework to promote cooperative autonomous decision-making of multi-UAV system. Through a pursuit–evasion game, a fixed-wing UAV can effectively maneuver to the rear of the opponent UAV and capture it. The key in this process is maneuver decision-making, and reinforcement learning shows great potential in this aspect.

In recent years, deep reinforcement learning has achieved significant accomplishments in various games, such as Alpha Go [5], OpenAI Five [6], AlphaStar for StarCraft [7], Suphx for Mahjong [8], and DeepNash for Stratego [9]. By modeling the pursuit–evasion game process as a Markov process, single-agent reinforcement learning has been applied to one-to-one pursuit–evasion games, such as approximate dynamic programming [10], Proximal Policy Optimization (PPO) [11], Deep Deterministic Policy Gradient (DDPG) [12], Twin Delayed Deep Deterministic Policy Gradient algorithm (TD3) [13], and Soft Actor Critic (SAC) [14]. A multi-UAV pursuit–evasion game is a scenario in which two or more UAVs engage in a game confrontation. Reinforcement learning methods are emerging as highly effective approaches to solving this problem [15].

Multi-agent reinforcement learning (MARL) has made remarkable progress in the field of multi-UAV collaboration and gaming. For a multi-UAV game, reference [16] proposed a MARL method based on the leadership–follower architecture, which improves the collaborative efficiency of multiple aircraft through hierarchical strategy optimization. However, this method fails to solve the problem of the sharp increase in training time in large-scale scenarios. Qu et al. improved the value decomposition mechanism of the QMIX algorithm, which enhanced the stability of multi-drone swarm games. However, in scenarios with more than 10 drones, the win rate still decreased significantly [17]. For multi-UAV pursuit and evasion, Lei et al. adopted an attention mechanism to enhance the credit allocation capability of MAPPO, alleviating the performance degradation caused by scale expansion, but did not consider the impact of flight trajectory oscillation on practical applications [18]. Although these latest studies have optimized the synergy effect in specific scenarios, they still have not simultaneously addressed the two core pain points of large-scale adaptability and flight stability.

The collaborative decision-making of heterogeneous UAVs/Unmanned Ground Vehicles (UGVs) in complex three-dimensional environments is also a current research hotspot. Reference [19] has realized a pursuit and evasion game between UAVs and UGVs in a three-dimensional complex environment. The communication constraints and autonomy framework design of multi-UAV systems are the key bottlenecks in their practical application. Recent studies have pointed out that communication delays and link interruptions in high-dynamic scenarios can lead to the failure of collaborative strategies, and the balance between distributed autonomy and centralized collaboration is the core to solving this problem [20]. Most of the existing methods optimize the performance of a single module

through communication fault-tolerant algorithms but lack robust design for the entire chain of collaborative decision-making—flight control. However, most of the existing heterogeneous collaborative research focuses on task division scenarios, with insufficient adaptability to high-dynamic pursuit and evasion game scenarios. The differences in maneuverability characteristics between fixed-wing and rotorcraft UAVs (such as speed and overload limits) make it difficult to balance the real-time performance and effectiveness of collaborative strategies.

Compared to the aforementioned games, the challenges encountered in multi-UAV pursuit–evasion games are as follows:

(1) Flight trajectory oscillation: Hierarchical RL is commonly used to decouple maneuver confrontation and flight control, but RL-based flight controllers often induce violent trajectory oscillations during high-overload engagements, restricting practicality.

(2) Poor scalability across UAV scales: Fixed network architectures of multi-agent RL (MARL) methods (e.g., MAPPO, QMIX) require retraining for different-scale scenarios, and their win rates and training efficiency degrade sharply as the number of UAVs increases, due to credit allocation issues and exponential growth in sample/training time.

In UAV pursuit–evasion game scenarios, UAVs must simultaneously master stable flight while learning to defeat opponents and ensure survival, significantly increasing the complexity of maneuver strategy acquisition. Consequently, hierarchical reinforcement learning approaches are proposed for pursuit–evasion game. These typically consist of a top-level tactical selector and a bottom-level controller [21–23]. Generally, the top-level strategy selects between offensive and defensive tactics [23] or chooses discrete maneuvers from a finite set [21], while the bottom-level strategy controls the maneuvering flight. A hybrid action hierarchical architecture is used in the literature [24], where the continuous layer controls maneuvers and the discrete layer assigns targets. A hierarchical structure is used in the literature [25], where each agent's virtual targets are generated by the upper layer and maneuver commands are generated by the lower layer.

Reinforcement learning controllers employed in these studies often focus solely on reaching desired states without considering flight path smoothness, frequently resulting in violent oscillations during high-overload engagements.

The second challenge lies in the applicability of deep reinforcement learning methods across scenarios with varying numbers of UAVs. Current multi-agent reinforcement learning (MARL) has demonstrated remarkable performance in multi-UAV cooperative pursuit–evasion games; the representative ones include the MAPPO algorithm [26] based on policy gradient and the QMIX algorithm [27] based on value decomposition. Reference [28] proposed a multi-agent reinforcement learning algorithm in response to the sparsity and instability of sample data in reinforcement learning algorithms, which demonstrated better training results. Reference [25] enhances win rates by applying hierarchical reinforcement learning to a multi-UAV coordinated game. While these papers demonstrate strong performance of multi-agent reinforcement learning, they require retraining for different-scale scenarios, limiting the method's generalizability. Reference [29] has developed a Gaussian-enhanced MARL framework for developing scalable avoidance strategies in dynamic tracking scenarios, but it can only be used for avoidance and not for tracking. Reference [30] addresses the scalability issue in multi-agent reinforcement learning by parameterizing actor and critic networks using attention-based neural network structures. However, it exhibits a significant decline in win rates as the scale increases. The monotonicity constraint of multi-agent reinforcement learning based on value decomposition fails when the scale increases, and the credit allocation variance of the multi-agent reinforcement learning algorithm based on policy gradient gradually increases with the increase in scale [31]. Additionally, all these methods suffer from the problem that training

time and sample size for multi-agent reinforcement learning increase exponentially with scale, limiting their application to large-scale pursuit–evasion game scenarios.

To address the two challenges outlined above, and effectively meet the protection requirements of civil airport airspace, this paper proposes a hierarchical collaborative pursuit–evasion game framework. It decomposes the multi-UAV collaborative pursuit–evasion game problem into three layers: target allocation, maneuver decision-making, and flight control. The target allocation layer performs dynamic target assignment. The maneuver decision-making layer generates maneuver commands using deep reinforcement learning based on the assigned targets. The flight control layer controls UAV flight according to the maneuver commands. The main contributions of this paper can be summarized as follows:

(1) A modular hierarchical framework that decomposes multi-UAV pursuit–evasion into target allocation, maneuver decision-making, and flight control layers, resolving the scalability bottleneck of MARL methods.

(2) A stable auxiliary gradient (SAG)-enhanced RL flight controller, which introduces angular acceleration constraints to eliminate trajectory oscillations without sacrificing control accuracy.

(3) A trajectory prediction-guided hierarchical RL maneuver decision-making method, using polynomial fitting to predict opponent positions and shape pre-tracking rewards, significantly improving one-on-one confrontation win rates.

(4) Dynamic target allocation algorithm based on situation advantage and threat level, with a dynamic value adjustment mechanism to achieve rapid, coordinated multi-target assignment and avoid resource concentration.

2. Problem Modeling

2.1. Game Model of Pursuit–Evasion with Multi-Fixed-Wing UAV

The scenario of this article is civil aviation airspace protection. There is an illegal UAV swarm intrusion, and our side needs to dispatch fixed-wing UAVs to engage in a pursuit–evasion game. The multi-fixed-wing UAV pursuit–evasion game refers to a scenario where two or more fixed-wing drones engage in maneuvering confrontation within space. By maneuvering around to the opponent's tail and pointing the nose at the opponent, they then use a capture net or other ways to capture the opponent's drone. During the process, it is necessary to simultaneously maneuver to prevent the opponent's drone from reaching the tail of one's own UAV. A schematic diagram of the game scenario is shown in Figure 1, where the orange cone represents the captured area of the UAV, and the green cone represents the capture area of the UAV. In this article, the range of the captured area is set to an angle of less than 20 degrees, the capture area is set to an angle of less than 4 degrees, and the distance is less than 4 km. The parameters are derived from the physical characteristics of fixed-wing unmanned aerial vehicles (UAVs) and the requirements of actual combat scenarios. Moreover, 4 km is the effective detection distance of the on-board sensors of small fixed-wing UAVs (such as electro-optical pods), 4° is the effective aiming angle for capture actions (such as launching capture nets), and 20° is the dangerous angle threshold for being locked by the enemy. These settings do not change with the number of drones, ensuring the consistency of the evaluation criteria in scenarios of different scales.

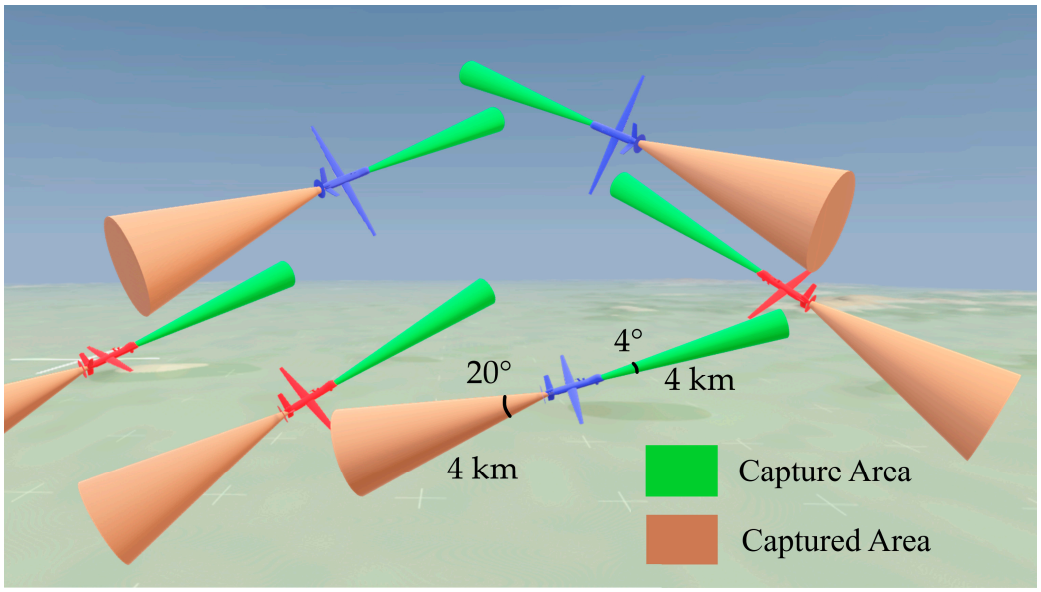


Figure 1. Schematic diagram of the game scenario (red UAVs are our side, blue UAVs are the opponent).

Note that the current study assumes noise-free state information (e.g., position, velocity, attitude) and no communication latency/loss. This simplification is intended to focus on validating the core hierarchical framework's scalability and cooperative decision-making effectiveness, isolating the performance of dynamic target allocation, trajectory prediction-guided maneuvering, and stable flight control from non-ideal environmental interference. This is a standard foundational research paradigm in multi-UAV system optimization [32,33], and the proposed framework is designed to support subsequent integration of robustness mechanisms for real-world deployment.

2.2. Fixed-Wing UAV Model

This paper adopts JSBSIM for aircraft simulation. JSBSIM is a nonlinear six-degrees-of-freedom aircraft simulation platform. Its authenticity has been verified by NASA. At present, JSBSIM has been widely applied in academic and industrial research [34].

$$\begin{aligned}\dot{x} &= u\cos\theta\cos\psi + v(\sin\phi\sin\theta\cos\psi - \cos\phi\sin\psi) + w(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi)\\ \dot{y} &= u\cos\theta\sin\psi + v(\sin\phi\sin\theta\sin\psi + \cos\phi\cos\psi) + w(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi)\\ \dot{z} &= u\sin\theta - v\sin\phi\cos\theta - w\cos\phi\cos\theta\end{aligned} \tag{1}$$

$$\begin{aligned}\dot{V} &= \frac{u\dot{u}+v\dot{v}+w\dot{w}}{V}\\ \dot{\alpha} &= \frac{u\dot{w}-w\dot{u}}{u^2+w^2}\\ \dot{\beta} &= \frac{\dot{v}V-v\dot{V}}{V^2\cos\beta}\end{aligned} \tag{2}$$

$$\begin{aligned}\dot{p} &= \frac{I_{xz}(I_x-I_y+I_z)pq+(I_zI_y-I_z^2-I_{xz}^2)qr+I_zL+I_{xz}N}{I_xI_z-I_{xz}^2}\\ \dot{q} &= \frac{M+(I_z-I_x)pr+(r^2-p^2)I_{xz}}{I_y}\\ \dot{r} &= \frac{I_{xz}L+I_zN+(I_x^2+I_z^2-I_xI_y)pq-I_{xz}(I_x-I_y+I_z)qr}{I_xI_z-I_{xz}^2}\end{aligned} \tag{3}$$

$$\begin{aligned}\dot{\phi} &= p+\tan\theta(r\cos\phi+q\sin\phi)\\ \dot{\theta} &= q\cos\phi-r\sin\phi\\ \dot{\psi} &= (r\cos\phi+q\sin\phi)/\cos\theta\end{aligned} \tag{4}$$

where ϕ, θ, ψ are the roll, pitch, and yaw flight attitude angles, respectively. p, q, r are the angular velocity in the body coordinate system. x, y, z are the aircraft position in the ground coordinate system. u, v, w are the velocity components in the x, y, and z directions, respectively. V, α, β are the velocity, angle of attack, and angle of sideslip, respectively.

Since this paper adopts JSBSIM, which includes actuator simulation, the actions output by the neural network in this paper are rudder deviation instructions. These instructions will pass through the actuator stage and then be input into the aircraft simulation model, which is also a manifestation of the high fidelity of JSBSIM. Moreover, due to the adoption of high-fidelity aerodynamics in the model simulation of JSBSIM, stall situations may also occur in flight control simulation. The impact of environmental stroke was not taken into account in this article.

The controls are $u = [\delta_e, \delta_a, \delta_r, \delta_T]$, including elevator, aileron, rudder command, and throttle command. The UAV coordinate system and angle definition are shown in Figure 2.

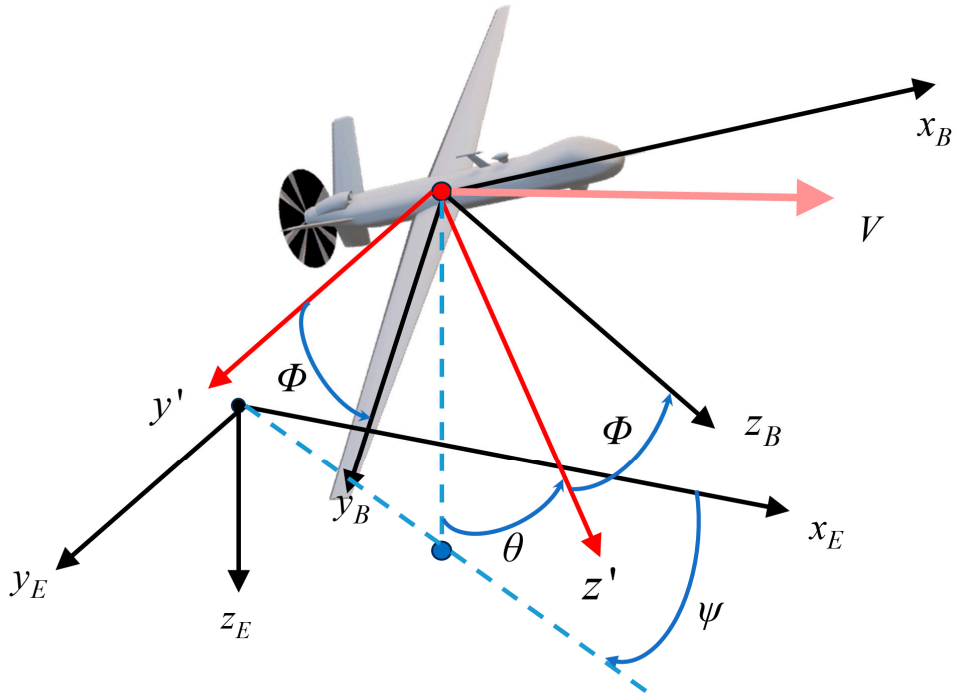


Figure 2. Modeling of aircraft coordinate systems and angles.

2.3. MDP Process

A Markov Decision Process is formulated by the tuple $< S, A, r, P, \rho_0, \gamma, T >$ [35]. Here, S denotes the state space, A denotes the action space, the transition probability function $P : S \times A \times S \to [0, 1]$ and the reward function $r : S \times A \to \mathbb{R}$ describe the dynamics of the environment, $\rho_0 : S \to [0, 1]$ describes the initial state distribution, T denotes the episode horizon, and $\gamma \in [0, 1)$ is the discount factor.

2.4. PPO Algorithm

In this paper, PPO [36] is employed as the foundational architecture for policy optimization. The optimal policy parameterized by PPO serves as the learning objective, achieved by maximizing the expected cumulative return $\mathcal{J}(\pi_\theta)$:

$$\mathcal{J}(\pi_\theta) = E[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)] \tag{5}$$

where $R(s_t, a_t)$ is the return. γ is the coefficient. A clipped alternative objective function is defined, which limits the maximum deviation between the old and new policies to maintain training stability of the policy network:

$$L^{clip}(\theta) = E_t[min(r_t(\theta)\hat{A}_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)] \tag{6}$$

where $r_t(\theta) = \pi_\theta / \pi_{\theta old}$ denotes the probability ratio, $\hat{A}_t$ represents the estimated advantage function, and ϵ is the hyperparameter that clips the ratio. This paper employs Generalized Advantage Estimation (GAE) to compute advantage estimation, defined as

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{7}$$

where $\delta_t = R_t + \gamma V(s_{t+1}) - V(s_t)$ represents the temporal difference (TD) error, and $V(s_t)$ is the value function output by the critic. The scaling factor $\lambda \in [0, 1]$ controls the bias–variance trade-off.

The critic network is trained to minimize the mean squared error (MSE) loss. MSE calculation is obtained by the mean of the errors of the target reward G_t and the predicted value $V_\phi(s_t)$.

$$\mathcal{J}^{critic}(\phi) = E_t[(G_t - V_\phi(s_t))^2] \tag{8}$$

An entropy bonus term for the policy is added to the objective function to enhance exploration. The optimization objective for the actuator network becomes

$$\mathcal{J}^{actor}(\theta) = L^{clip}(\theta) + \sigma H(\pi_\theta) \tag{9}$$

where σ is the hyperparameter for scaling the entropy contribution.

3. Hierarchical Maneuvering Decision-Making Algorithm

The hierarchical decision-making algorithm proposed in this paper mainly consists of three layers, namely, the target allocation layer, the maneuvering decision-making layer, and the aircraft control layer, thereby solving the maneuvering decision-making problem of the scalable multi-UAV cooperative pursuit–evasion game. The overall framework diagram is shown in Figure 3.

This paper adopts a hybrid control mode of centralized optimization at the target allocation layer and distributed decision-making at the execution layer. The target allocation layer conducts centralized target allocation based on global situation information to ensure coordination. The maneuvering decision-making layer and flight control layer adopt distributed reinforcement learning to enhance the real-time response speed.

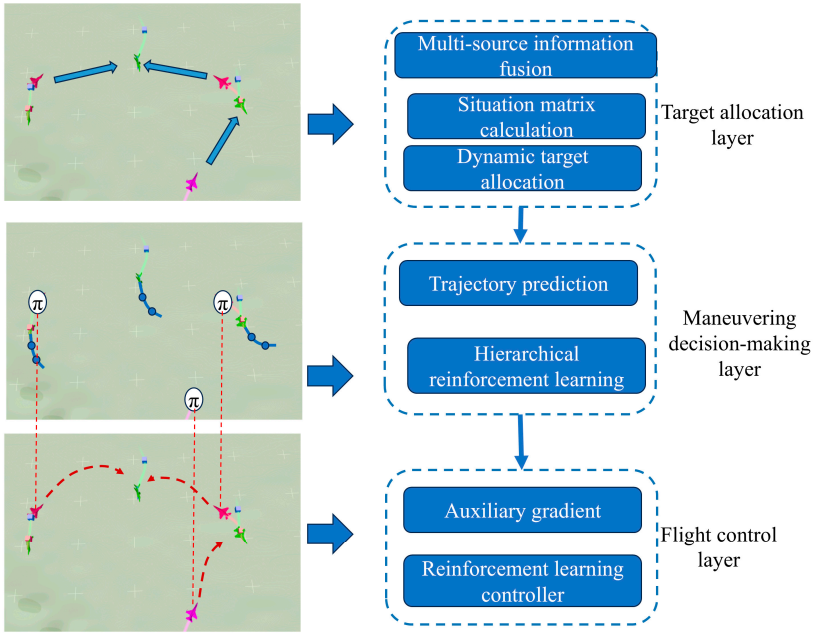


Figure 3. Hierarchical collaborative pursuit–evasion game framework.

3.1. Reinforcement Learning Aircraft Controller with Stable Auxiliary Gradients

The core objective of this module is to ensure that the UAV can accurately respond to the pursuit–evasion game instructions when performing tasks such as airspace protection at civil airports, while maintaining smooth flight and avoiding secondary risks caused by trajectory oscillations.

3.1.1. Observation Space and Action Space

(1) Observation space

The observation space established in this paper primarily includes the deviation between the desired state variables and the current state variables, as well as the current flight state variables of the UAV. These mainly comprise Δh altitude difference, $\Delta\psi$ heading difference, ΔV velocity difference, h current flight altitude, $\sin\phi$ sin of roll angle, $\cos\phi$ cos of roll angle, $\sin\theta$ sin of pitch angle, $\cos\theta$ cos of pitch angle, (Vx, Vy, Vz) velocity components along the three axes and velocity magnitude V, and angular velocities p, q, r, etc.

(2) Action Space

The action space includes continuous variables for pitch rudder deflection, yaw rudder deflection, roll rudder deflection, and throttle $\delta_a, \delta_e, \delta_r \in [-1, 1], \delta_t \in [0, 1]$.

3.1.2. Reward Function

Expected value error rewards are proposed as follows.

Define the altitude error reward as $r_h = e^{-(\Delta h / s_h)^2}$

where Δh is the difference between the expected altitude and the current altitude, and s_h is the scale factor for altitude error.

Define the heading error reward as $r_\psi = e^{-(\Delta\psi / s_\psi)^2}$

where $\Delta\psi$ is the difference between the expected heading and the current heading, and s_ψ is the scale factor for heading error.

Define the velocity error reward as $r_V = e^{-(\Delta V / s_V)^2}$

where ΔV is the difference between the expected velocity and the current velocity, and s_V is the scale factor for velocity error.

The overall reward is $r_{error} = (r_h \cdot r_\psi \cdot r_V)^{1/3}$.

3.1.3. Stabilizing the Auxiliary Gradient

(1) Smoothness measurement

This paper formally presents a practical metric for measuring the oscillation degree of the unmanned aerial vehicle (UAV) flight trajectory under a given strategy π.

The most commonly used quantitative measures are longitudinal acceleration [37], lateral acceleration [38], and jerk [39]. Unlike autonomous vehicles, unmanned aerial vehicles travel in three-dimensional space, and their trajectory oscillations are mainly caused by continuous changes in flight attitudes. A smooth trajectory should maintain a stable angular velocity.

Therefore, in this paper, angular acceleration is adopted as the key metric for this smoothness. In this case, the one-step smoothness metric (OSSM) $\kappa(s, a)$ is defined as the norm of angular acceleration:

$$\kappa(s, a) = \|\alpha\| = \left\| \frac{d\boldsymbol{\omega}}{dt} \right\| \tag{10}$$

where $\boldsymbol{\omega} = [p, q, r]^T$ is the angular velocity vector and α is the angular acceleration.

The multi-step smoothness metric (MSSM) is defined as the superposition of the one-step smoothness metric, and an auxiliary network $\mathcal{H}(s, a)$ is used to approximate it:

$$\mathcal{H}(s, a) = \mathbb{E}\left[\sum_{k=0}^{T} \gamma^k \kappa(s_{t+k}, a_{t+k}) \right] \tag{11}$$

It is constrained to be less than a certain upper bound $\mathcal{K}_t$.

The objective of the auxiliary network is to accurately predict cumulative angular acceleration. Therefore, its loss calculation $\mathcal{J}^{Aux}(\psi)$ is carried out by predicting the difference between multi-step angular acceleration and the actual value.

$$\mathcal{J}^{Aux}(\psi) = \mathbb{E}_t \left[\left(\left[\sum_t \gamma^t \kappa(s_t, a_t) \right] - \mathcal{H}_\psi(\boldsymbol{s}_t) \right)^2 \right] \tag{12}$$

(2) Three-layer nested optimization problem

We aim to maximize the cumulative rewards while meeting the smoothness constraints:

$$\max_\pi \mathbb{E}\left[\sum \gamma^k r(s, a) \right] \text{ s.t. } \mathbb{E}\left[\sum_{k=0}^{T} \gamma^k \kappa(s_{t+k}, a_{t+k}) \right] \leq \mathcal{K}_t \tag{13}$$

Construct the Lagrange function, introduce the Lagrange multiplier $\hat{\lambda}$ ($\hat{\lambda} \geq 0$, physical meaning: the penalty coefficient for violating the smoothness constraint), and transform it into an unconstrained optimization problem through the Lagrange relaxation method.

$$\min_{\hat{\lambda} \geq 0} \max_\theta \left[\mathcal{J}(\pi_\theta) - \hat{\lambda} \left(\mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \gamma^t \kappa(\boldsymbol{s}_t, \boldsymbol{a}_t) \right] - \mathcal{K}_t \right) \right] \tag{14}$$

Since the $\mathcal{J}(\pi_\theta)$ (value function) and $\mathcal{H}(s, a)$ need to be learned from the data, the problem is ultimately decomposed into a three-layer nested optimization problem, with the following objectives for each layer:

Inner layer (max): $\max_\theta (L^{clip}(\theta) + \sigma H(\pi_\theta))$, the strategy strives to maximize the cumulative reward.

Middle layer (min): $\min_{\theta}(\mathcal{H}_{\psi}(\boldsymbol{s}, \pi_{\theta}(\boldsymbol{s})) - \mathcal{K}_t)$, the strategy strives to control the estimated cumulative angular acceleration to keep it below the upper bound of the constraint.

Outer layer (min): $\min_{\hat{\lambda}\geq 0}\mathbb{E}\left[-\hat{\lambda}(\kappa_{\psi}(\boldsymbol{s}, \pi_{\theta}(\boldsymbol{s})) - \mathcal{K}_t)\right]$, adjust the penalty coefficient, increase when the constraint is violated, and decrease when the constraint is satisfied.

Under the PPO framework, the above objective is approximated through the proxy loss function. The trajectories sampled by the current policy are utilized to avoid resampling for each update. Finally, the loss function of the PPO-SAG actor network is obtained:

$$\mathcal{J}(\pi_{\theta}, \psi) = L^{clip}(\theta) + \sigma H(\pi_{\theta}) - \hat{\lambda}\left(\mathbb{E}_{(\boldsymbol{s},\boldsymbol{a})\sim\rho_{\pi}}\left[\kappa_{\psi}(s, a)\right] - \mathcal{K}_t\right) \tag{15}$$

(3) Gradient calculation and parameter update

The three-layer optimization objective is achieved by alternately updating the policy parameters θ, auxiliary network parameters ψ, and Lagrange multipliers $\hat{\lambda}$ through the gradient descent method.

(a) Major Gradient

Optimize the policy π_{θ} using the PPO algorithm to maximize cumulative rewards:

$$\nabla_{\theta}\mathcal{J}_{\pi}(\theta) = \nabla_{\theta}(L^{clip}(\theta) + \sigma H(\pi_{\theta})) \tag{16}$$

(b) Stable Auxiliary Gradient

By using the chain differentiation rule, the stable auxiliary gradient can be obtained as

$$\nabla_{\theta}\mathcal{J}^{Aux} = -\hat{\lambda}\cdot\mathbb{E}\left[\nabla_{a}\mathcal{H}_{\psi}(\boldsymbol{s}, \boldsymbol{a})\Big|_{\boldsymbol{a}=\pi_{\theta}(\boldsymbol{s})}\cdot\nabla_{\theta}\pi_{\theta}(\boldsymbol{s})\right] \tag{17}$$

The auxiliary gradient is used to penalize those actions that cause high-angular acceleration.

(c) Lagrange multiplier update

Since the outer layer is a min problem, it needs to be updated in the opposite direction of the gradient, and the formula is

$$\hat{\lambda} \leftarrow \hat{\lambda} - \eta_{\lambda}\nabla_{\hat{\lambda}}\mathcal{J} = \hat{\lambda} + \eta_{\lambda}\mathbb{E}\left[\mathcal{H}_{\psi}(\boldsymbol{s}, \pi_{\theta}(\boldsymbol{s})) - \mathcal{K}_t\right] \tag{18}$$

where η_{λ} is the learning rate of the multiplier. If $\mathcal{H}_{\psi} > \mathcal{K}$ (constraint violation), $\hat{\lambda}$ will increase; otherwise, it will decrease.

The overall training framework is shown in the following Figure 4. The pseudo-code of the SAG-PPO algorithm is shown in Algorithm 1.

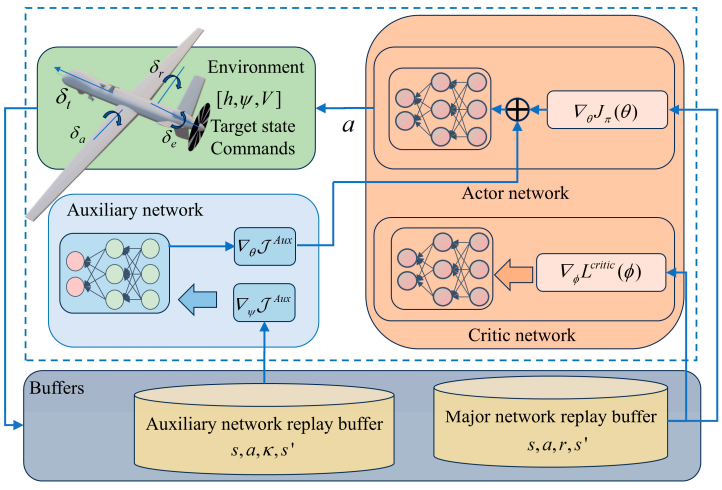


Figure 4. The architecture of a stable gradient-assisted reinforcement learning flight control network.

Algorithm 1: SAG-PPO:

Initialize the policy network π_θ and the value network V_ϕ
Initialize the auxiliary network $\mathcal{H}_\psi$
Initialize the Lagrange multiplier $\lambda = 0.1$
Initialize the hyperparameters and the upper bound $\mathcal{K}_t$ of smoothness
for step = 1 to Max-steps do
    env step
    get (s, a, r, s', κ, done) and store them in buffer
    If reach the buffer length
        calculate critic loss $\mathcal{J}^{critic}(\phi) = E_t[(G_t - V_\phi(s_t))^2]$
        calculate the gradient $\nabla\mathcal{J}^{critic}(\phi)$ and update critic parameters
        calculate the main loss $\mathcal{J}(\pi_\theta) = L^{clip}(\theta) + \sigma H(\pi_\theta)$
        calculate the loss of the auxiliary network $J^{Aux}(\theta) = -\hat{\lambda}\left(\mathbb{E}_{(\boldsymbol{s},\boldsymbol{a})\sim\rho_\pi}\left[\kappa_\psi(\boldsymbol{s},\boldsymbol{a})\right] - \mathcal{K}_t\right)$
        $\mathcal{J}(\pi_\theta,\psi) = \mathcal{J}(\pi_\theta) + J^{Aux}(\theta)$ total loss is obtained
        calculate the gradient $\nabla\mathcal{J}(\pi_\theta,\psi)$ and update actor parameters
        calculate the gradient $\nabla_\psi\mathcal{J}^{Aux}$ and to update the auxiliary network parameters
        $\hat{\lambda} \leftarrow \hat{\lambda} - \eta_\lambda\nabla_{\hat{\lambda}}\mathcal{J} = \hat{\lambda} + \eta_\lambda\mathbb{E}\left[\mathcal{H}_\psi(\boldsymbol{s},\pi_\theta(\boldsymbol{s})) - \mathcal{K}_t\right]$ Update the Lagrange multiplier

The PPO-SAG controller adopts a fully connected network (hidden layer size (128, 128)), with an input dimension of 14 (observation space: height difference, course difference, angular velocity, etc.) and an output dimension of 4 (control instructions: roll, pitch, yaw, throttle), forward propagation time $O(D_{in} \times D_{hidden} + D_{hidden} + D_{hidden} \times D_{out})$ (D_{in} = 14, D_{hidden} = 128, D_{out}= 4), and the output time of a single control command for a single unmanned aerial vehicle is approximately 0.8 ms. The stable auxiliary gradient and Lagrange multiplier update are only performed during the training phase. During the inference phase, the trained policy output instructions are directly called, with no additional computational overhead.

3.2. Maneuver Decision-Making Layer Based on Trajectory Prediction and Hierarchical Reinforcement Learning

In the scenario of civil airport airspace protection, this layer provides precise pursuit guidance or evasion decisions for security UAVs by predicting the trajectories of illegally invading UAVs, thereby enhancing the effectiveness of protection tasks. The overall framework is shown in Figure 5.

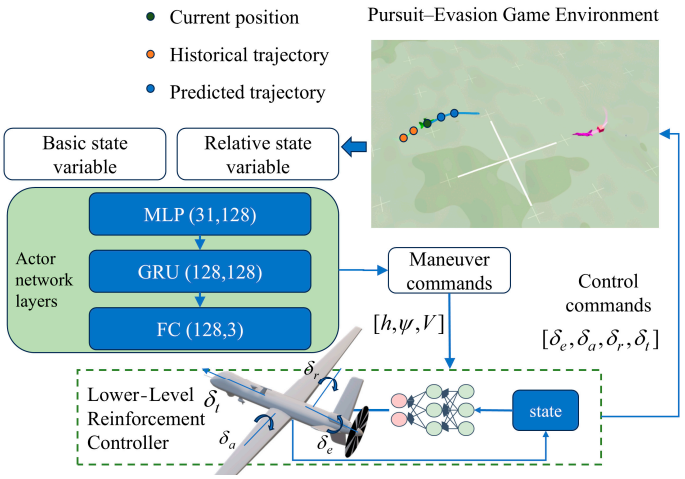


Figure 5. Schematic diagram of maneuvering layer based on trajectory prediction and hierarchical reinforcement learning.

3.2.1. Trajectory Prediction

By predicting the movement trend of the opponent UAV, the position of the opponent UAV after a certain period of time can be obtained, thereby forming a pre-tracking state. (Figure 6 presents a schematic diagram of the pre-tracking.) This can effectively increase the win rate. Therefore, this paper proposes a prediction of opponent UAV positions based on polynomial fitting.

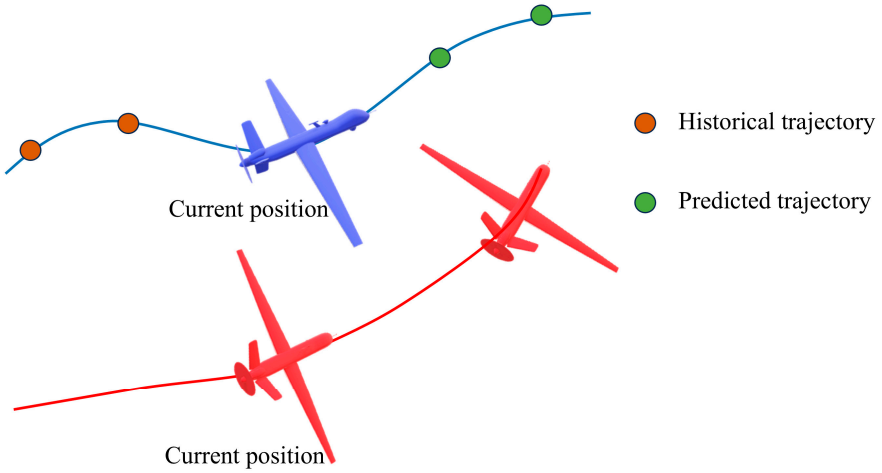


Figure 6. Schematic diagram of the pre-tracking (red UAVs is our side, blue UAVs is the opponent).

Since the movement of opponent UAV satisfies the basic motion equations, it is impossible for sudden changes to occur in a short period of time. Therefore, it is feasible to predict the trajectory by using the polynomial fitting method.

$$\begin{bmatrix} x_{ep}(t) \\ y_{ep}(t) \\ z_{ep}(t) \end{bmatrix} = P_{3\times3} \begin{bmatrix} t^2 \\ t \\ 1 \end{bmatrix} = \begin{bmatrix} p_1^x & p_2^x & p_3^x \\ p_1^y & p_2^y & p_3^y \\ p_1^z & p_2^z & p_3^z \end{bmatrix} \begin{bmatrix} t^2 \\ t \\ 1 \end{bmatrix} \tag{19}$$

$$\begin{cases} p_1^x = \frac{1}{2\cdot\Delta t^2}(x_e(t) - 2\cdot x_e(t-\Delta t) + x_e(t-2\cdot\Delta t)) \\ p_2^x = \frac{1}{2\cdot\Delta t^2}(3\cdot x_e(t) - 4\cdot x_e(t-\Delta t) + x_e(t-2\cdot\Delta t)) \\ p_3^x = x_e(t) \end{cases} \tag{20}$$

$$\begin{aligned} x_e(t+\Delta t) &= p_1^x\cdot\Delta t^2 + p_2^x\cdot\Delta t + p_3^x \\ y_e(t+\Delta t) &= p_1^y\cdot\Delta t^2 + p_2^y\cdot\Delta t + p_3^y \\ z_e(t+\Delta t) &= p_1^z\cdot\Delta t^2 + p_2^z\cdot\Delta t + p_3^z \\ V_{ex}(t+\Delta t) &= \frac{x_e(t+\Delta t)-x_e(t)}{\Delta t} \\ V_{ey}(t+\Delta t) &= \frac{y_e(t+\Delta t)-y_e(t)}{\Delta t} \\ V_{ez}(t+\Delta t) &= \frac{z_e(t+\Delta t)-z_e(t)}{\Delta t} \end{aligned} \tag{21}$$

where $x_e(t)$, $x_e(t-\Delta t)$, and $x_e(t-2\cdot\Delta t)$ represent the X-axis positions of the other party at the current moment, the moment before Δt, and the moment before $2\cdot\Delta t$, respectively. $x_e(t+\Delta t)$ represents the predicted X-axis position of the target at the moment after Δt.

3.2.2. Observation Space and Action Space

(1) Observation space

This paper has already carried out target allocation at the upper level. Therefore, UAVs take the observed space variables as input and generate action output through

strategic model reasoning. This method extracts features from battlefield information and normalizes them, taking the basic state of our UAV and the geometric state of both sides' UAV information as input.

Specifically, the basic state variables include the UAV's altitude h, the magnitude of velocity in the x direction V_x, the magnitude of velocity in the y direction V_y, the magnitude of velocity in the z direction V_z, the angular velocity q, the magnitude of velocity V, and the rate of velocity change ΔV. The relative geometric state information mainly includes the state of the opponent UAV at the current moment $state_{t_0} = [\Delta x_{t_0}, \Delta y_{t_0}, \Delta z_{t_0}, \theta^{t_0}_{Los_body}, \psi^{t_0}_{Los_body}, V^{t_0}_x, V^{t_0}_y, V^{t_0}_z]$, the predicted state of the opponent UAV $state_{t_0+\Delta t}$ after time Δt, and the predicted state of the opponent UAV $state_{t_0+2\Delta t}$ after time $2\Delta t$, where $\theta_{Los_body}, \psi_{Los_body}$ are the pitch and heading of the line of sight vector in the aircraft coordinate system. To achieve the tactical behavior of pre-tracking, we combine the predicted position of the opponent UAV with the current position information as the input. The observed variables and their definitions are shown in Table 1.

Table 1. Observation space.

Variable Type	Variable Symbol	Meaning
Basic state variables	h	Height
	V_x	Velocity value in the x direction
	V_y	Velocity value in the y direction
	V_z	Velocity value in the z direction
	V	Magnitude of speed
	ΔV	Change in velocity
	q	Pitch Angle velocity
Relative geometric state variables	$state_{t_0}$	Current status of opponent aircraft
	$state_{t_0+\Delta t}$	Status of opponent UAV after Δt time
	$state_{t_0+2\Delta t}$	Status of opponent UAV after $2\Delta t$ time

(2) Action space

Since this paper employs hierarchical reinforcement learning, in the upper layer, $\Delta\psi, \Delta h, \Delta V$ changes in heading, altitude, and speed are used as actions. Each action component is five discrete decision spaces, shown in Table 2.

Table 2. Action space.

Action Variables	Scope
Δh (km)	$\{-0.2, -0.1, 0, 0.1, 0.2\}$
$\Delta\psi$ (radian)	$\{-\pi/6, -\pi/12, 0, \pi/12, -\pi/6\}$
ΔV (mach)	$\{-0.2, -0.1, 0, 0.1, 0.2\}$

3.2.3. Reward Shaping

(1) Situation reward

In reference [40], angular situation and distance situation were considered separately. However, in the context of this paper, to meet the termination conditions, both angular and distance conditions need to be satisfied simultaneously. Therefore, this paper introduces a comprehensive consideration of distance and angular situation.

Angular situation: In order to guide the UAV towards the opponent's UAV, relatively strict angular reward conditions were adopted.

$$Ta = \begin{cases} 10 & if\ q_{Los} < 4/180 \times pi \\ 1 + 2\frac{15/180 \times pi - q_{Los}}{15/180 \times pi} & elseif\ 4/180 \times pi < q_{Los} < 15/180 \times pi \\ 1 - \frac{\alpha - 15/180 \times pi}{35/180 \times pi - 15/180 \times pi} & elseif\ 15/180 \times pi < q_{Los} < 35/180 \times pi \\ 0 & \text{else} \end{cases} \tag{22}$$

$$Td = \begin{cases} 1, d \le d_A \\ e^{1-d/d_A}, d > d_A \end{cases} \tag{23}$$

The distance situation is considered as follows: when the distance is less than the termination distance of the UAV, the distance situation is 1; when it is less than the termination distance, the distance situation value decays.

By coupling the angular situation and the distance situation, taking into account the opponent UAV threat, the situation reward can be obtained as

$$R_{adv} = \omega_1 \times Ta \times Td - \omega_2 \times Ta_e \times Td \tag{24}$$

where Ta_e represents the angle situation value of the opponent UAV, with ω_1 set to 1 and ω_2 set to 0.7.

(2) Pre-tracking reward

This is a direct incentive for the agent to perform pre-tracking. Directly encourage the agent to aim the nose of the aircraft at the future position of the opponent UAV rather than its current position. This is a direct means to increase the win rate. By calculating the LOS angle, a reward is given, where q_{Los} is the LOS angle and σ is $4/180 * pi$. Through the $q_{Los}^{t_0}, q_{Los}^{t_0+\Delta t}, q_{Los}^{t_0+2\Delta t}$ calculation and weighting, the result can be obtained:

$$r_{lead} = 0.5e^{-(q_{Los}^{t_0} - \sigma)} + 0.3e^{-(q_{Los}^{t_0+\Delta t} - \sigma)} + 0.2e^{-(q_{Los}^{t_0+2\Delta t} - \sigma)} \tag{25}$$

(3) Win/lose rewards

$$r_{win-lose} = \begin{cases} 5 & , win \\ -5 & , lose \end{cases} \tag{26}$$

3.2.4. Hierarchical Reinforcement Learning Algorithm

During the upper-level training process, the low-level model network parameters are frozen, and the high-level model network parameters are updated. During the training process, the action output of the upper-level agent serves as an observation for the lower-level agent. The maneuvering decision algorithm based on hierarchical PPO is shown in Algorithm 2.

The pseudo-code of the hierarchical PPO algorithm is as follows.

Algorithm 2: hierarchical PPO

Load low level model M_{low} and set it to eval mode
Initialize actor, critic parameters, and hyperparameters
for step = 1 to Max-steps do
    get action from actor
    transfer action to low level model
    M_{low} output action to env
    env step
    get (s, a, r, s', κ, done) and store them in buffer
    If reach the buffer length
        calculate critic loss $\mathcal{J}^{critic}(\phi) = E_t[(G_t - V_\phi(s_t))^2]$
        calculate the gradient $\nabla\mathcal{J}^{critic}(\phi)$ and update critic parameters
        calculate actor loss $\mathcal{J}(\pi_\theta) = L^{clip}(\theta) + \beta H(\pi_\theta)$
        calculate the gradient $\nabla\mathcal{J}(\pi_\theta)$ and update actor parameters

In this network layer, polynomial fitting is adopted to predict the future position of the enemy, and a linear least squares problem is solved based on K historical trajectory points, with a time complexity of O (K^3). In practice, K is taken as 3, which is equivalent to a constant time of O (1), and the time consumption for a single prediction is approximately 0.5 ms.

Hierarchical reinforcement learning inference The structure of the upper-level policy network (MLP + GRU) is fixed (hidden layer size [128, 128]), independent of the scenario scale. The input feature dimension is 31, the time complexity of forward propagation is $O(D_{in} \times D_{mlp} + D_{mlp} \times D_{gru} + D_{gru}^2 + D_{gru} \times D_{hidden} + D_{hidden} \times D_{out})$ (D_{in} = 31, D_{hidden} = 128, D_{mlp} = 128, D_{gru} = 128, D_{out} = 3), and the reasoning time for a single unmanned aerial vehicle is approximately 1.2 ms. The reasoning of N drones can be executed in parallel, so the time consumption is the same as that of a single drone.

3.3. Dynamic Target Allocation Algorithm Based on Dynamic Value Adjustment

In response to the demand for multi-security UAVS to collaboratively protect the airport airspace, this algorithm rationally allocates the intervention tasks for illegal UAVs through dynamic value adjustment. It avoids the concentration or waste of resources and ensures the comprehensiveness and efficiency of protection.

3.3.1. Situation Assessment

Based on the game theory algorithm, we have established a pursuit–evasion game situation assessment model. In the northeast coordinate system of the earth coordinate system, assuming the position of the i-th unmanned aerial vehicle (UAV) on the red side is (x_{ri}, y_{ri}, z_{ri}) and its velocity vector is $\vec{V}_{ri} = (Vx_{ri}, Vy_{ri}, Vz_{ri})$, and the position of the j-th UAV on the blue side is (x_{bj}, y_{bj}, x_{bj}) and its velocity vector is $\vec{V}_{bj} = (Vx_{bj}, Vy_{bj}, Vz_{bj})$, the relative position vector can be obtained as $\vec{D} = (x_{bj} - x_{ri}, y_{bj} - y_{ri}, z_{bj} - z_{ri})$. According to the geometric model of the pursuit–evasion game situation, the relative entry angle between the red UAV and the blue UAV can be calculated as

$$A_\alpha^{ij} = \arccos\left(\frac{\vec{D}\cdot\vec{V}_{ri}}{\left\|\vec{D}\right\|\left\|\vec{V}_{ri}\right\|}\right),\ A_\alpha^{ji} = \arccos\left(\frac{-\vec{D}\cdot\vec{V}_{bj}}{\left\|\vec{D}\right\|\left\|\vec{V}_{bj}\right\|}\right) \tag{27}$$

The advantage of the red UAV over the blue UAV is defined based on the angle of entry as

$$A^{ij} = \frac{\pi - A_{\alpha}^{ij}}{\pi} \tag{28}$$

Similarly, the threat level of the blue drone to the red drone is defined as

$$A^{ji} = \frac{\pi - A_{\alpha}^{ji}}{\pi} \tag{29}$$

Thus, the situation advantage of the red party UAV over the blue party UAV is obtained as follows:

$$S^{ij} = A^{ij} - A^{ji} \tag{30}$$

According to the game theory method, $-S^{ij}$ represents the situation advantage of the blue UAV over the red UAV.

3.3.2. Dynamic Target Allocation Method

According to the above situation calculation method, the situation matrix can be obtained as

$$\begin{bmatrix} S^{11} & S^{12} & \dots & S^{1j} \\ S^{21} & S^{22} & \dots & S^{2j} \\ \dots & \dots & \dots & \dots \\ S^{i1} & S^{i2} & \dots & S^{ij} \end{bmatrix} \tag{31}$$

When allocating targets, it is necessary to consider both the situation advantage of our aircraft over the opponent and the threat posed by the opponent to our aircraft. Therefore, by integrating the situation advantage ($S_{ij} > 0$) and the threat ($S_{ij} < 0$), the formula is

$$U_{ij} = \alpha \cdot \max(S_{ij}, 0) + \beta \cdot \max(-S_{ij}, 0) \tag{32}$$

where α represents the advantage weight ($\alpha > 0$), indicating the degree of importance attached to the situation advantage, fixed at 1; β represents the threat weight ($\beta > 0$), indicating the degree of importance attached to the threat.

Then the target allocation model can be constructed as

$$\begin{aligned} &\max \sum_{i=1}^{N_a} \sum_{j=1}^{N_t} U_{ij} \cdot x_{ij} \\ &\text{s.t.} \sum_{j=1}^{N_t} x_{ij} = 1 \ \forall j \in [1, \dots, N_a] \\ &\quad x_{ij} \in \{0, 1\} \end{aligned} \tag{33}$$

where $x_{ij} \in \{0, 1\}$ represents the decision variable. When it is 1, it indicates that the target j is assigned to UAVi; when it is 0, it indicates that it is not assigned to UAVi. The optimization target is $\max \sum_{i=1}^{N_a} \sum_{j=1}^{N_t} U_{ij} \cdot x_{ij}$. Since each UAV in this article can only be assigned one target, the constraint condition is $\sum_{j=1}^{N_t} x_{ij} = 1 \ \forall j \in [1, \dots, N_a]$. However, many of our UAVs can jointly pursuit the same opponent UAV. Therefore, in this paper, there is no constraint that the target can only be assigned to one of our UAVs. Thus, this problem cannot be solved by the traditional auction algorithm. If the matrix game method is used to transform it into a hybrid game problem, the number of our strategies is 6^6, which is too large and the solution time is too long, and it cannot meet the real-time requirements. Therefore, this paper proposes the use of lightweight collaborative allocation rules.

This paper proposes a dynamic value adjustment mechanism to reduce the attraction of a friendly UAV to other friends when it chooses to pursue an opponent UAV, avoid excessive concentration, and naturally incentivize the selection of suboptimal but unsaturated targets.

The procedure is as follows:

1. Initialization: Calculate the value matrix U based on the original situation matrix S.
2. Find the global best pairing: Locate the current maximum value $U[i^*, j^*]$ in U.
3. Allocation: Allocate opponent UAV j^* to friendly UAV i^*.
4. Dynamic devaluation: Reduce the attractiveness of opponent aircraft j^* to all other friendly UAVs: $U[i, j^*] = U[i, j^*]^* \delta, i \in [1, \ldots N_a]$, where δ is the depreciation factor.
5. Update matrix: Set $U[i^*, j]$ to the minimum value (or mark it as assigned) to prevent duplicate selection.
6. Repeat steps 2 to 5 until all friendly machines have been allocated or there are no available targets.

By devaluing the selected target, it encourages other friendly UAVs to seek new targets. This algorithm is highly efficient in calculation: in each round, it only needs to find the maximum value of the matrix ($O(N^2)$), iterate N times, and the total complexity is $O(N^3)$, with extremely high real-time performance. Figure 7 shows the dynamic target allocation process.

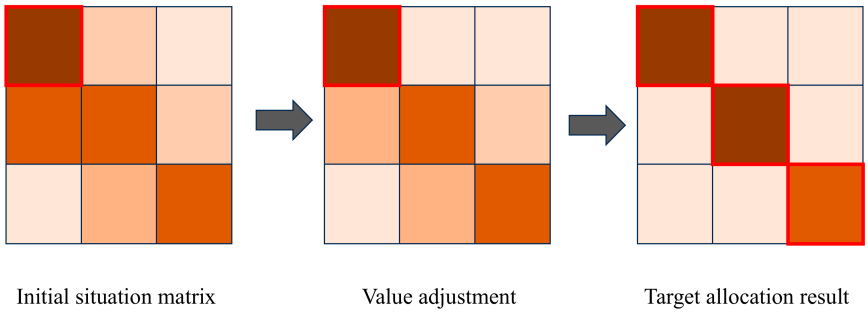


Figure 7. Dynamic target allocation process.

Theorem 1. *The dynamic target allocation algorithm proposed in this paper is a 1/2 approximation algorithm for the maximum weight matching problem. That is, the total utility value of the solution obtained by the algorithm satisfies* $w(M) \geq \frac{1}{2} w(M^*)$, *where* M^* *is the global optimal solution.*

Proof: We formalize the many-to-many objective assignment problem as a Maximum Weight Matching problem. □

Bipartite Graph: Construct a bipartite graph $G = (R, B, E)$, where R is the set of red drones (friendly aircraft), B is the set of blue drones (opponent aircraft), and E is the edge set, where each edge (i, j) connects $i \in R$ to $j \in B$. The weight of each edge (i, j) is $w_{ij} = U_{ij}$, which is the defined utility value that integrates the situation advantage and threat. All weights $w_{ij} > 0$.

We will prove it by comparing the greedy solution X with the optimal solution X. For each edge X in the optimal match, it must "conflict" with at most two edges in the greedy match X.

If (i, j) is also selected by the greedy algorithm, then it exists simultaneously in M and M^*.

If (i, j) is not selected, it must be because during the operation of the greedy algorithm, when edge (i, j) still has a chance to be selected, the algorithm selects another edge with a greater weight, which is connected to i or j.

Specifically, there might be an edge $(i,k) \in M$ (that is, the blue side k is eventually assigned to the red side i), and $w_{ik} > w_{ij}$.

Or there might be an edge $(l,j) \in M$ (that is, the blue side j is eventually allocated by the red side l), and $w_{lj} > w_{ij}$.

Since a friendly machine can only be assigned one target and an opponent machine can be assigned multiple times but not repeatedly from the same friendly machine, an edge can "block" at most pairs of edges in the optimal solution. Therefore, $\sum_{(i,j)\in M^*} w_{ij} \leq \sum_{(p,q)\in M} 2w_{pq}$, which is equivalent to $w(M) \geq \frac{1}{2}w(M^*)$.

The algorithm in this paper adds a dynamic devaluation mechanism on the basis of the standard greedy algorithm. This operation will not undermine the proof of the 1/2 approximation ratio mentioned above. The devaluation operation is carried out only after one edge has been added to the match. The proof process focuses on the decision made when the algorithm selects edges, and this decision is based on the weights at that time. Devaluation only affects subsequent choices. Since we always choose the edge with the current maximum weight, and after devaluation, the weights of some edges become smaller, this may actually make it less likely for the algorithm to make "poor" choices in subsequent selections, and it may even obtain a better solution than the standard greedy algorithm. Therefore, the quality of the solution of the algorithm in this paper will at least not be worse than that of the standard greedy algorithm.

$$w(M_{\text{ours}}) \geq w(M_{\text{greedy}}) \geq \frac{1}{2}w(M^*) \tag{34}$$

4. Simulation Results

Using a single machine (AMD Ryzen 9 9800X CPU 4.5 GHz, 24-core, one NVIDIA GeForce RTX 3090 24 G GPU, 128 GB RAM), all experiments were run three times using different random seeds. In this article, the important parameter settings of the PPO algorithm are listed in Table 3.

Table 3. Parameter settings of PPO algorithm.

Name	Value
n_rollout_threads	32
use_valuenorm	True
hidden_sizes	[128, 128]
activation_func	relu
gain	0.01
use_recurrent_policy	True
recurrent_n	1
learn_rate	0.0005

4.1. Simulation Comparison of Flight Control Layer

In the flight control layer, the PPO algorithm is adopted to train the flight control. The expected state instructions are given every 12 s, and the network is sampled and trained from the process. Since the update of the policy network takes into account the influence of both the primary gradient and the auxiliary gradient simultaneously, this paper compares the training effect of using the auxiliary network with that of not using it.

The training results of the rewards curve are shown in Figure 8. From the overall trend of the data, it can be seen that the performance with/without the stable auxiliary gradient is roughly the same. This study indicates that adding the proposed smooth auxiliary gradient does not significantly reduce learning efficiency.

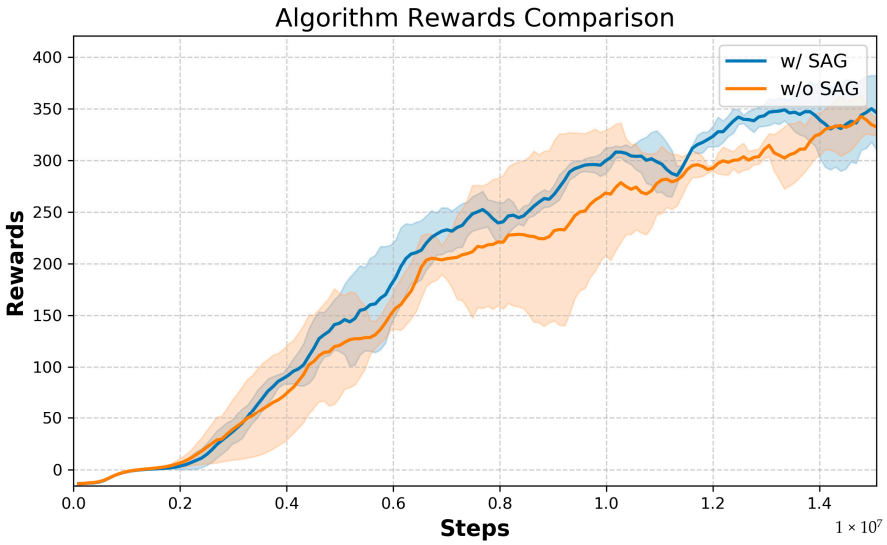
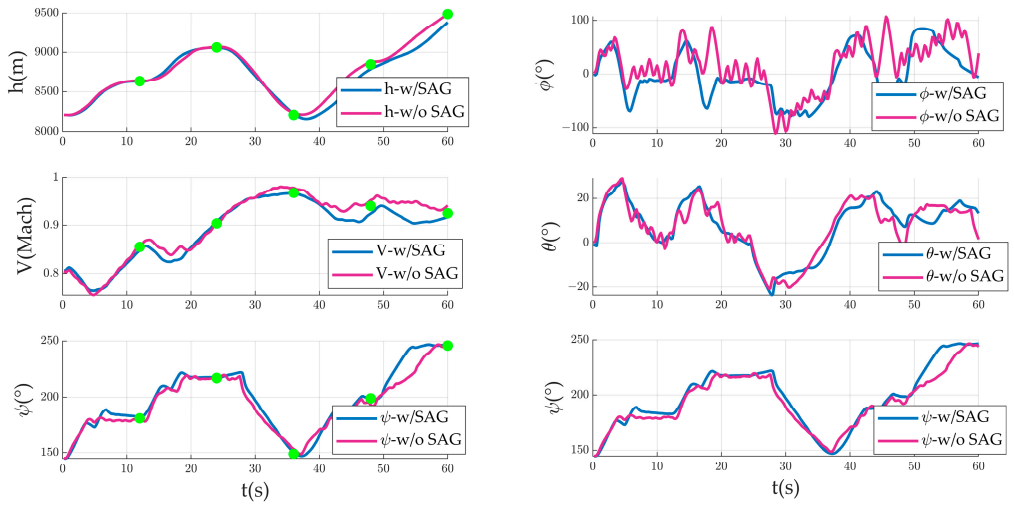


Figure 8. The reward curve with and without SAG.

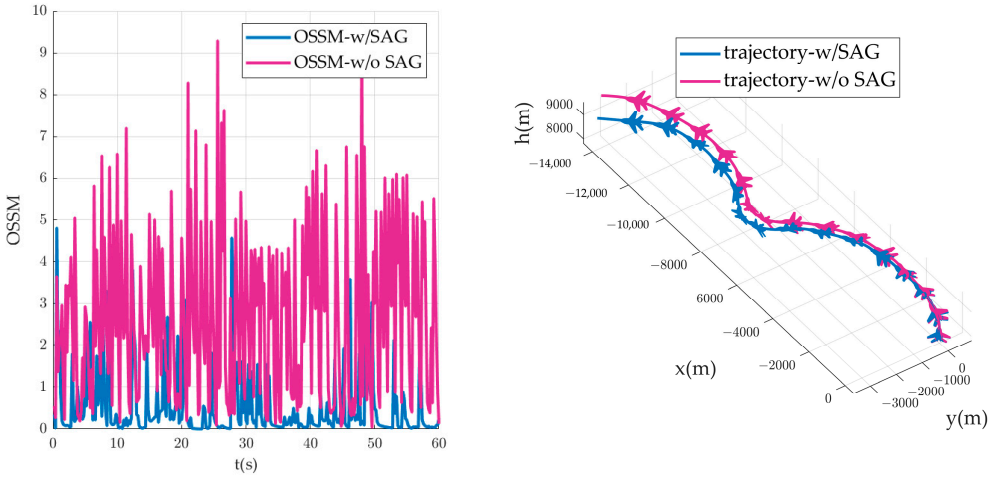
In addition, to prove the effectiveness of the auxiliary gradient proposed in this paper, the instruction responses of the trained network were compared, and an expected state instruction was given every 12 s.

Figure 9a shows the curves of the expected state and the actual flight state. The green dots in Figure 9a represent the expected states. It can be seen from the figure that both with and without the auxiliary network can reach the expected state within the specified time, indicating that the auxiliary network does not affect the state tracking effect of the main network. Figure 9b shows the variation curve of the UAV attitude angles, from which it can be seen that the attitude angle oscillation is relatively intense without the auxiliary network. Figure 9c shows the OSSM variation curve, from which it can be seen that the auxiliary network has demonstrated an excellent OSSM suppression effect. Figure 9d shows the three-dimensional flight trajectory, and the two are quite similar.



(**a**) (**b**)

Figure 9. *Cont.*



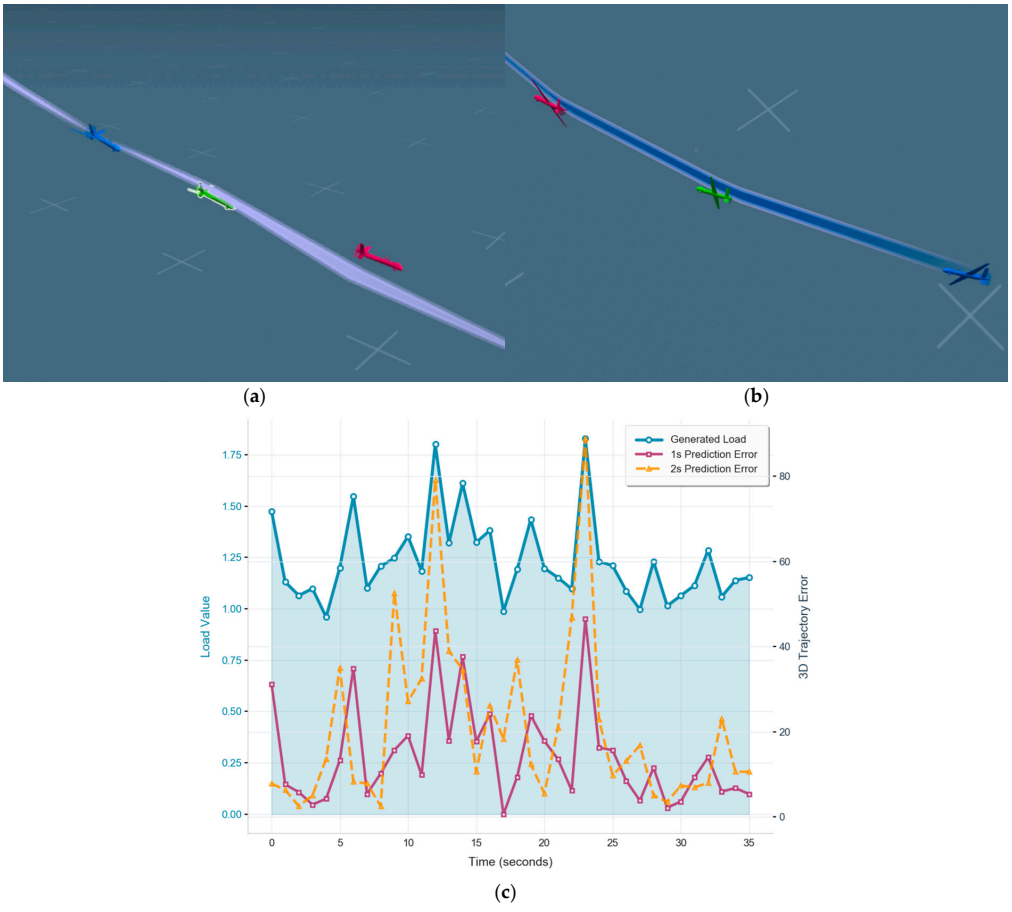
(**c**) (**d**)

Figure 9. Comparison chart of flight state responses with and without SAG. (**a**) State response curve; (**b**) Flight attitude curve; (**c**) OSSM variation curve; (**d**) Three-dimensional flight trajectory.

4.2. Simulation Comparison of Maneuvering Decision-Making Layer

4.2.1. Trajectory Prediction Simulation

In this section, we quantitatively evaluate the accuracy of the polynomial trajectory prediction used in this paper. We selected a flight trajectory for online trajectory prediction and used the root mean square error of position as the evaluation index. The results of the trajectory prediction are shown in Figure 10.



(**a**) (**b**)

(**c**)

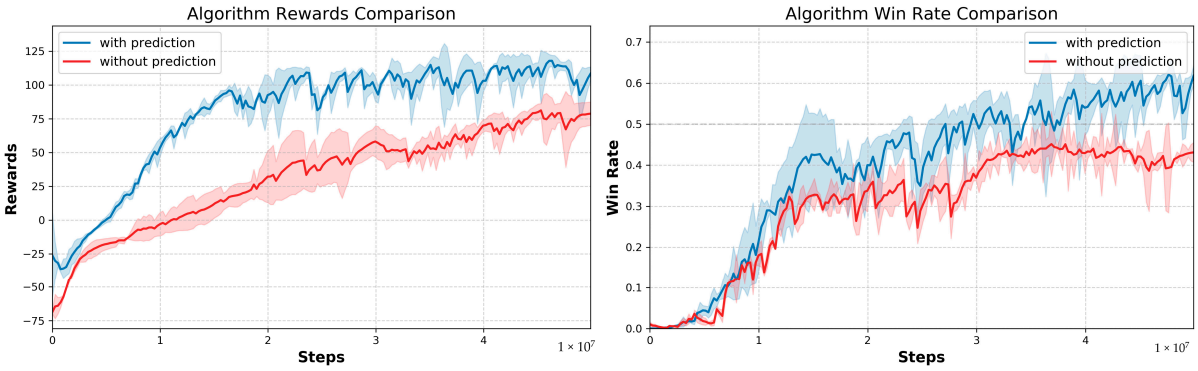
Figure 10. Simulation results of polynomial trajectory prediction. (**a**) The 9 s trajectory prediction; (**b**) The 26 s trajectory prediction; (**c**) Trajectory prediction error and overload curve.

In Figure 10a,b, the blue UAV represents the position of the UAV at the current moment, the green UAV represents the predicted position 1 s later, and the red UAV represents the predicted position 2 s later. It can be seen from this that the polynomial trajectory prediction method in this paper has a relatively high prediction accuracy. Figure 10c shows the variation curve of trajectory prediction error and overload over time. It can be seen from this that when the overload of UAV flight changes significantly, the trajectory prediction error will increase, but the maximum error can be maintained within 100 m.

4.2.2. The Training Results of Hierarchical Reinforcement Learning

In the maneuver decision-making layer, the red UAV is driven by hierarchical reinforcement learning and the blue UAV is driven by rules. Ablation experiments were conducted at the maneuvering decision-making level. By comparing the hierarchical reinforcement learning method with and without trajectory prediction, the effectiveness of trajectory prediction was demonstrated.

Figure 11a shows the rewards curve. It can be seen that with prediction, there is a faster convergence speed, and the rewards it eventually converges to are larger than those without prediction. Figure 11b shows the win rate curve. Eventually, the algorithm with prediction converges to around 0.6 and the algorithm without prediction converges to around 0.43. Therefore, it can be proved that trajectory prediction has a significant effect on improving the win rate of pursuit–evasion game. In addition, to demonstrate the effectiveness of the trained agents, the confrontation process between the final trained red agent and the rule-driven blue agent is presented.



(**a**) (**b**)

Figure 11. Comparison of the training reward and win rate curves with and without trajectory prediction. (**a**) rewards; (**b**) win rate.

Figure 12 provides screenshots of the confrontation process in the tacview visualization software. Initially, the red side and the blue side face each other head-on, with a distance of 26.4 km between the two sides. In the subsequent 70 s and 93 s, the red and blue sides adopt similar maneuvers, forming a single loop. Then, at 143 s, the red side clearly gains a situation advantage through trajectory prediction. On the 266 s and 390 s, the red side demonstrates obvious pre-tracking tactical behavior. Finally, at 546 s, the red side reaches the termination condition, and the red side wins. It can be seen from the process that the red side can effectively utilize trajectory prediction to gain a situation advantage.

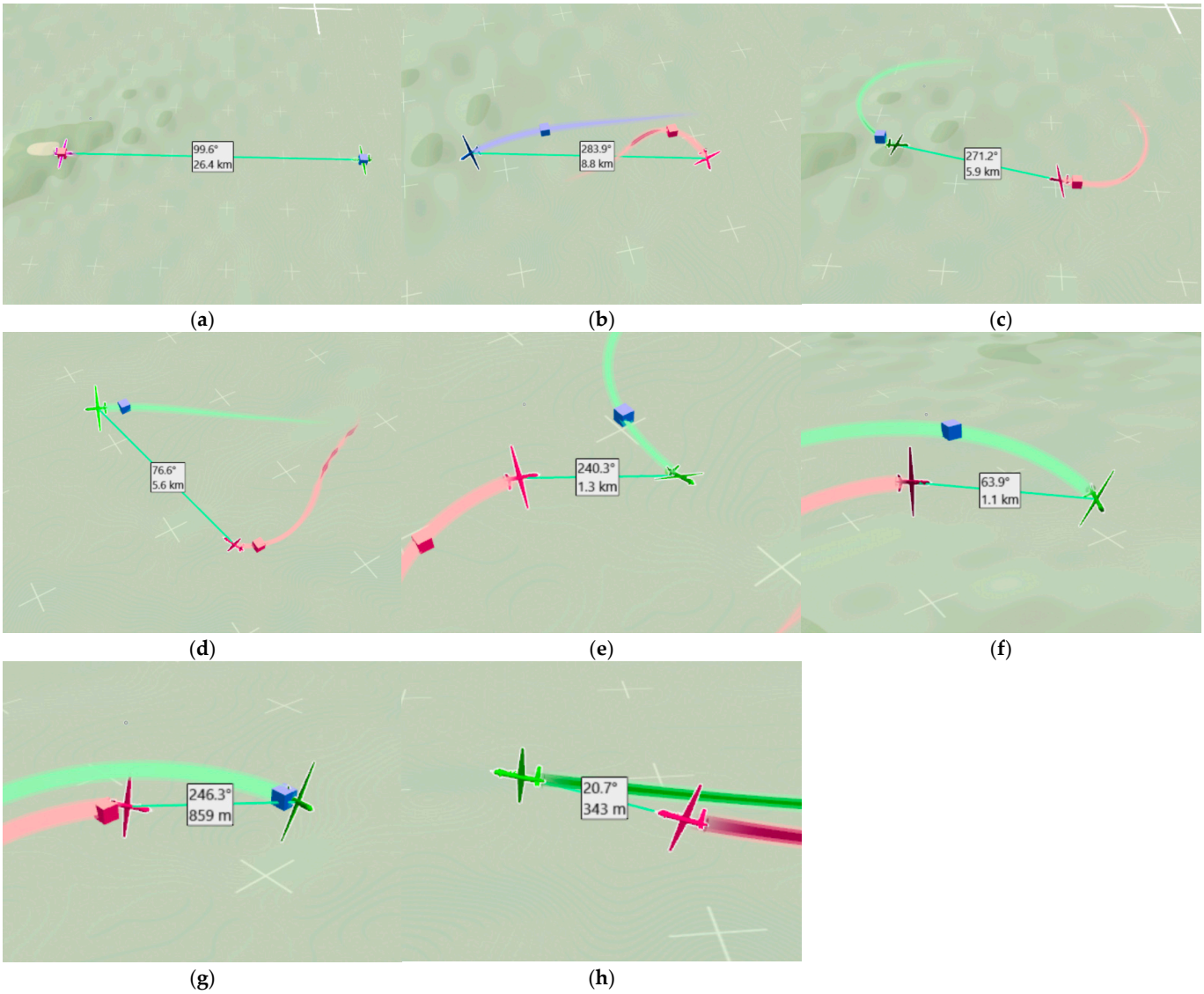
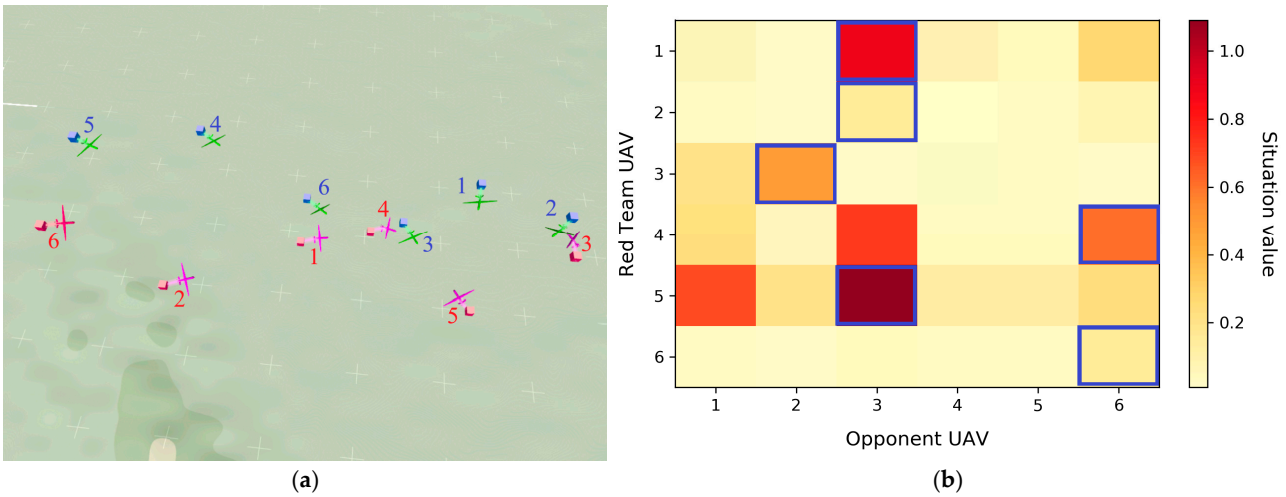


Figure 12. Screenshot of the 1V1 pursuit–evasion game confrontation process. (**a**) 0 s; (**b**) 70 s; (**c**) 93 s; (**d**) 143 s; (**e**) 215 s; (**f**) 266 s; (**g**) 390 s; (**h**) 546 s.

4.3. Target Allocation Simulation

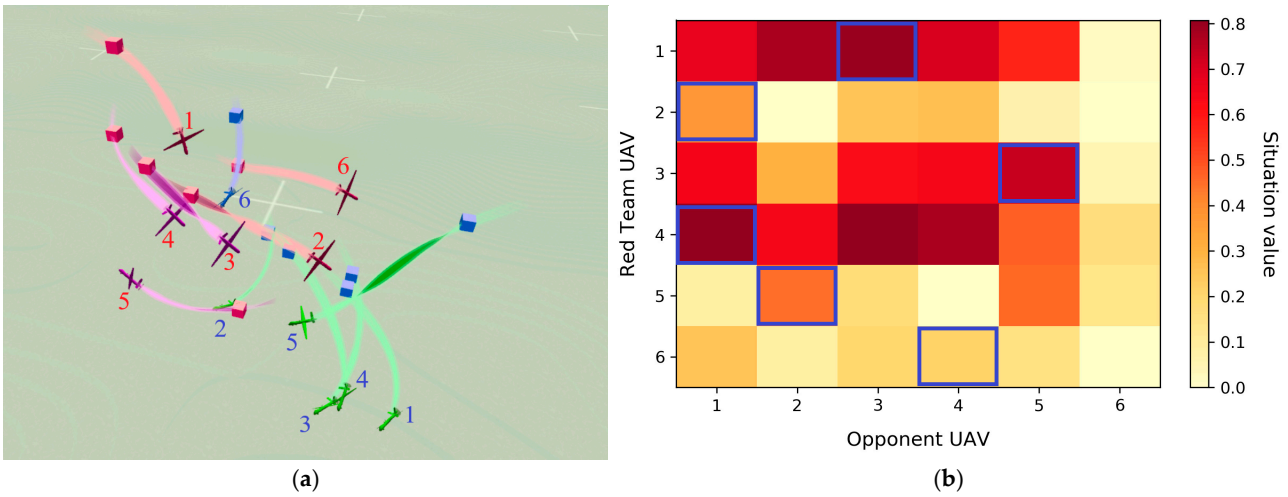
To demonstrate the target allocation process, taking 6V6 as an example, the heat maps of the target allocation utility matrix U for 35 s and 156 s are presented, and the target allocation results are labeled. The results are shown in Figures 13 and 14.

Figures 13a and 14a show the three-dimensional situation of the UAV at that time, while Figures 13b and 14b present the target allocation utility matrix U and the allocation results. It can be seen from this that the target allocation algorithm proposed in this paper can effectively allocate targets based on the situation, and the proposed dynamic devaluation mechanism avoids allocating the same target to a certain extent. For example, in Figure 14b, the target allocated by the red UAV6 is the blue UAV4, rather than the blue UAV1. Meanwhile, the allocation method proposed in this paper can also generate allocation focus. For instance, in Figure 13b, the red UAV1, UAV2, and UAV5 are all allocated as the blue UAV3. This enables the production of a coordinated pursuit effect. It has a better synergy effect compared with the previous fixed one-to-one allocation model.



(**a**) (**b**)

Figure 13. The result of target allocation in the 6V6 scenario (35 s). (**a**) Three-dimensional situation; (**b**) Situation matrix heat map.



(**a**) (**b**)

Figure 14. The result of target allocation in the 6V6 scenario (136 s). (**a**) Three-dimensional situation; (**b**) Situation matrix heat map.

This paper also analyzes the hyperparameters in target allocation. Parameter optimization includes situation weight optimization and target devaluation factor optimization.

Situation weight optimization: In order to obtain the optimal situation weight factor parameters, this paper compares the influence of different situation weight factors on the win rate. Experiments were conducted on different weight factors in three scenarios, namely, 3V3, 6V6, and 9V9, and the win rate values under different weight factors were obtained. We took the β values as 0.4, 0.5, 0.6, 0.7, 0.8, and 0.9.

As can be seen from Figure 15, when the situation weight factor is 0.7, it has the highest win rate. The reason is that when the situation weight factor is too high, the agent tends to adopt defensive tactics and is difficult to form a coordinated offensive strategy; thus, the win rate is not high. When the situation weight factor is too low, the agent ignores the threat of an opponent UAV and is easily lost.

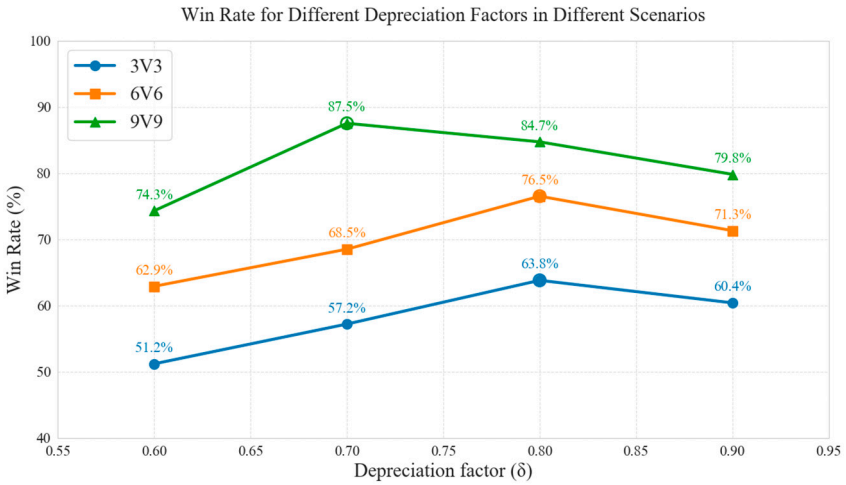


Figure 15. The win rate curves under different situation weight factors.

Target devaluation factor optimization: The devaluation factor reduces the focusing effect among multiple UAV and encourages friendly UAVs to seek other targets. However, if the value is too small, it may make it difficult to form a coordinated tactical effect and reduce the win rate. Therefore, this paper optimizes the devaluation factor δ, with values of 0.6, 0.7, 0.8, and 0.9.

As can be seen from Figure 16, when the scale increases, it is even more necessary to reduce the target depreciation factor, thereby prompting friendly UAVs to search for other targets. Taking all factors into consideration, the δ is taken as 0.8.

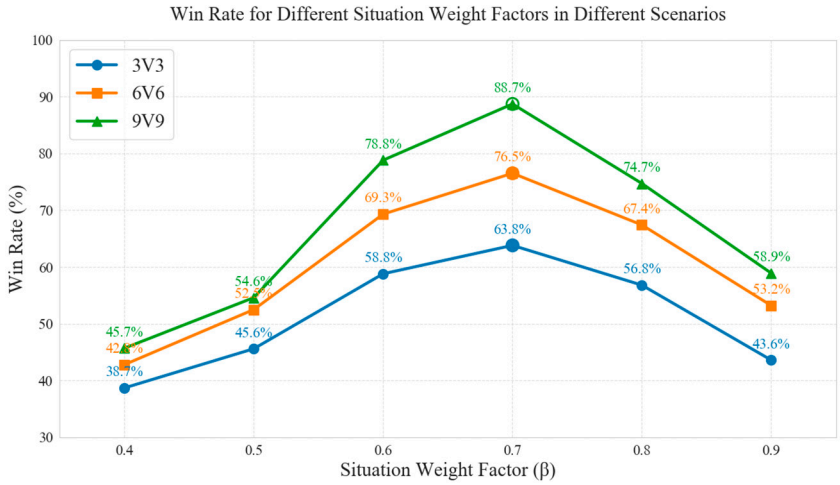


Figure 16. The win rate curves under different target depreciation factors.

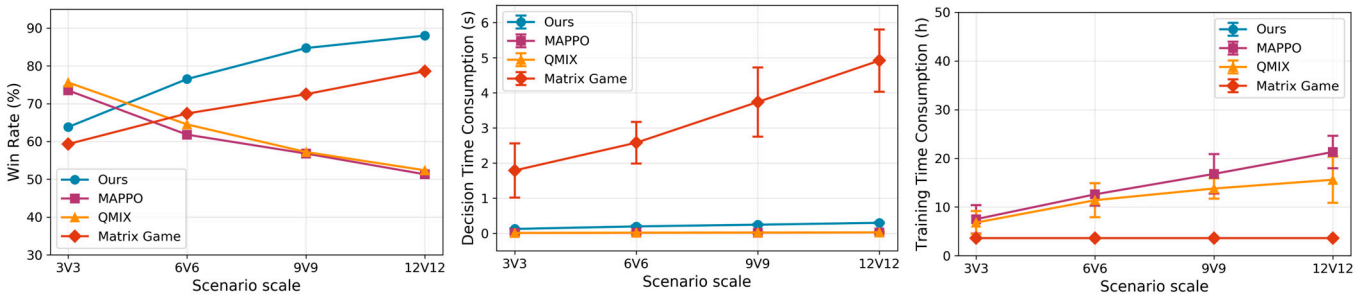
4.4. Overall Algorithm Comparison

To demonstrate the generalization of the algorithm proposed in this paper in a multi-UAV pursuit–evasion game at different scales, simulations were conducted at 3V3, 6V6, 9V9, and 12V12 scales. The comparison algorithms include MAPPO, QMIX, and the matrix game [41]. References [42,43] demonstrate that these two algorithms have achieved good results in multi-UAV pursuit–evasion game. The MAPPO and QMIX algorithms adopt the maneuvering decision-making method of hierarchical reinforcement learning in reference [43]. The red side uses the above-mentioned algorithms, while the blue side uses the rule-based target allocation method. The rule-based target allocation method and the matrix game target allocation algorithm only differ from the algorithm proposed in this paper at the target allocation layer. The same algorithm is used at both the maneuvering decision-making layer and the flight control layer. Among them, the rule-based target

allocation method is as follows: Initially allocate the target closest to one's own side. After merging with the opponent aircraft, switch the target to this one.

Three evaluation indicators were selected: win rate, decision-making time, and training time consumption. Thus, the advantages and disadvantages of the algorithm can be examined from multiple aspects. Under the same initial conditions, 30 Monte Carlo simulations were conducted to obtain the win rates of each algorithm.

Figure 17 shows the comparison results of the method proposed in this paper with the MARL method and the matrix game algorithm. From Figure 17a, the win rate of our method and the matrix game algorithm increases as the scale increases, while the win rate of the MARL algorithm decreases as the scale increases. This is related to the credit allocation problem that the MARL method faces in large-scale game scenarios. In the case of small-scale game scenarios, the QMIX algorithm has the highest win rate. When the scale increases, our method has the highest win rate.



(**a**) (**b**) (**c**)

Figure 17. Algorithm comparison under multiple indicators. (**a**) Win rate; (**b**) Decision-making time; (**c**) Training time consumption.

As shown in Figure 17b, the MARL method has the shortest single-step decision-making time consumption, followed by our method. The matrix game optimization target allocation algorithm takes a relatively long time and struggles to meet the real-time requirements of the pursuit–evasion game. Moreover, as the scale increases, the time consumption of the matrix game optimization target allocation algorithm increases sharply. As can be seen from Figure 17c our method does not require retraining with changes in scale, while the training time of MARL algorithms increases sharply with the increase in scale.

To demonstrate the reliability of 30 simulation times, we conducted more simulation times to obtain the changes in the win rate, thereby proving the effectiveness of the number of simulation times in the evaluation of algorithm quality.

As shown in Figure 18, the win rate difference between 30 and 40 simulations is $\leq 2.8\%$ (average 1.9%), and between 30 and 50 simulations is $\leq 3.5\%$ (average 2.3%). A two-sample t-test ($\alpha = 0.05$) was conducted to verify the difference between win rates of 30 and 50 simulations. The results show p-values > 0.05 for all algorithms, indicating no statistically significant difference, confirming that 30 Monte Carlo simulations are sufficient for reliable performance evaluation.

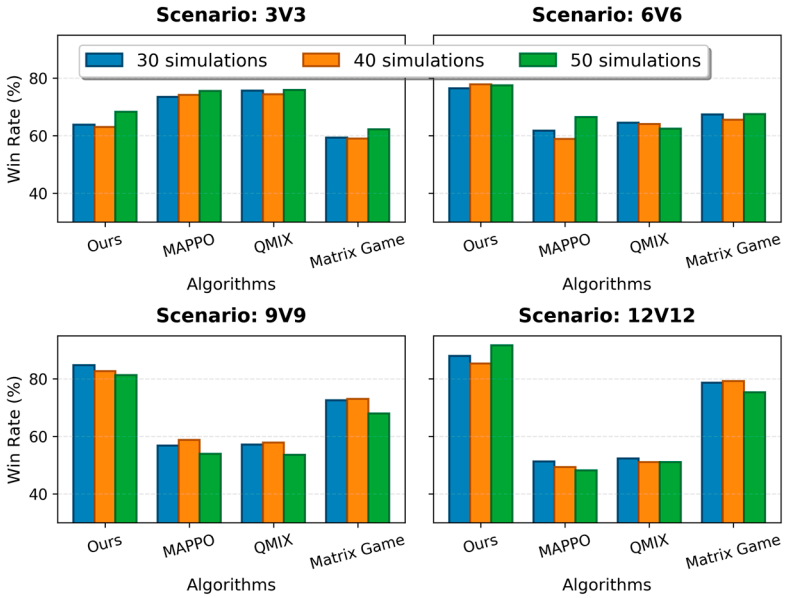


Figure 18. The win rate changes with different Monte Carlo simulation times.

The blue opponents in the above simulation results are generally rule-driven. Therefore, in order to better demonstrate the advantages of the method proposed in this paper over other methods, we will compare the method proposed in this paper with the MAPPO, QMIX, and matrix game methods. Conduct 30 Monte Carlo confrontations on a scale of 3V3, 6V6, 9V9, and 12V12 to obtain the win rate, which is presented in Figure 19.

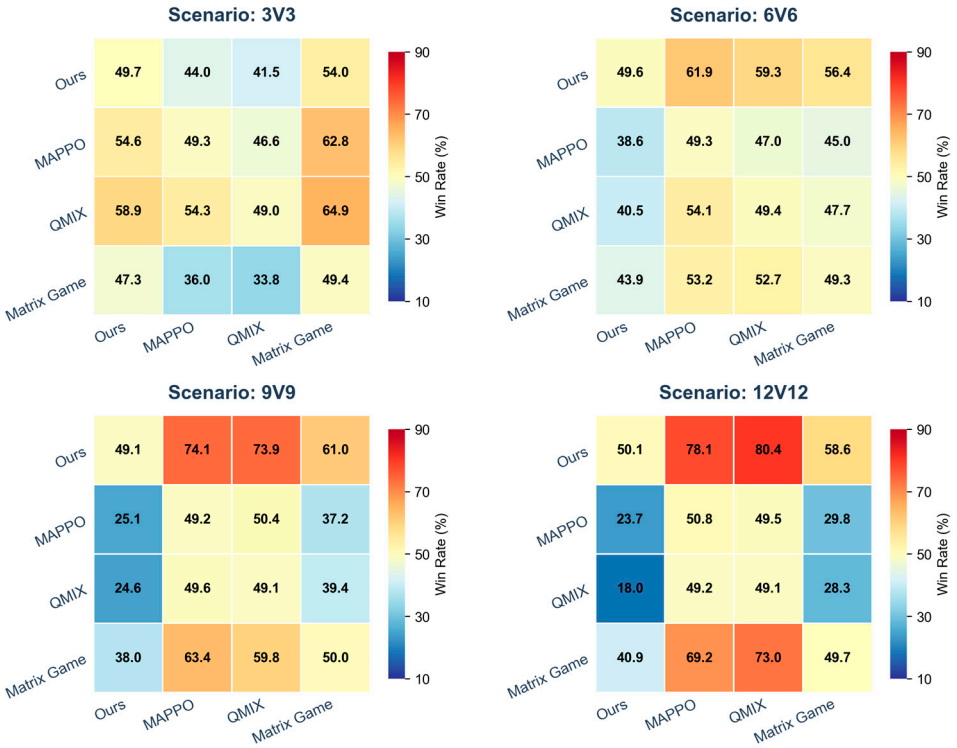


Figure 19. Heat map of the game confrontation win rate matrix.

The elements of the matrix heat map in Figure 19 represent the win rates of the game between the method in the row and the method in the column. As can be seen from Figure 19, the method proposed in this paper does not have an advantage over the MAPPO, QMIX, and matrix game methods when the scale is relatively small. However, as the scale

increases, the win rate gradually rises. When playing against MAPPO and QMIX in 12V12, the win rate can reach about 80%.

The experimental results show that although MAPPO and QMIX have an advantage in decision-making time consumption, their training time consumption far exceeds that of our method, and their win rate in large-scale scenarios is significantly lower than that of the hierarchical allocation method proposed in this paper. This proves the advantages of our method in terms of training efficiency and large-scale scalability. The matrix game target allocation algorithm obviously cannot meet the requirements of real-time decision-making. Due to the existence of allocation delay during target allocation, the win rate is not as high as that of our method.

To conduct a detailed analysis of the confrontation process, this article provides screenshots of the 3V3 confrontation process as shown in Figure 20.

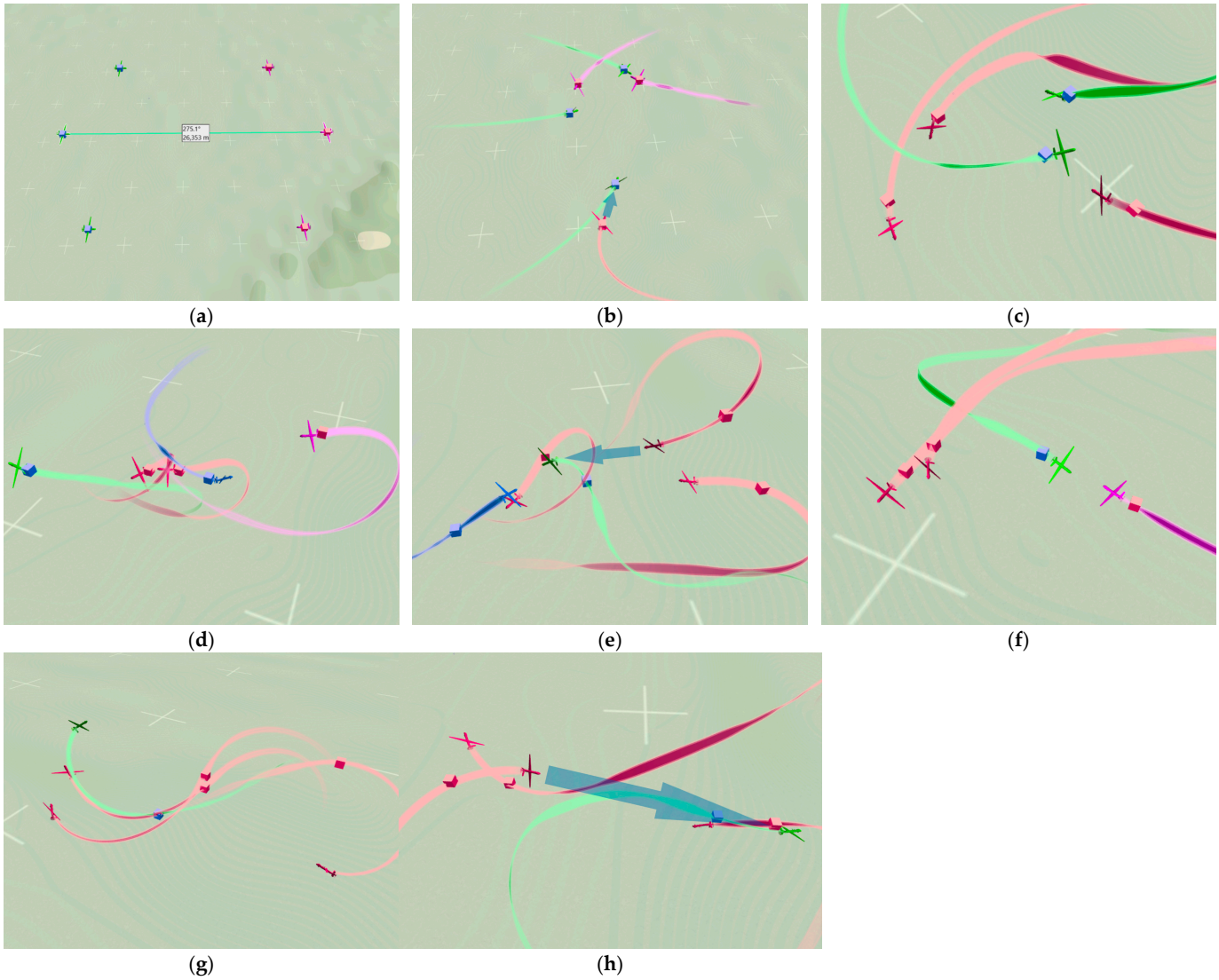


Figure 20. The 3V3 pursuit–evasion game process of this paper's method vs. rule-based method. (**a**) 0 s; (**b**) 40 s; (**c**) 90 s; (**d**) 150 s; (**e**) 253 s; (**f**) 300 s; (**g**) 360 s; (**h**) 392 s.

The above figure shows the 3V3 simulation confrontation process. In Figure 20a, at the initial moment, the red and blue sides formed a formation and faced each other head-on. In Figure 20b, a UAV in the blue square reaches the termination condition because it ignores the threat behind the tail. Then, at 253 s and 392 s, the other two drones on the blue side meet the termination conditions. It can be seen from the confrontation process that the

entire confrontation process is significantly shortened compared to the 1V1 confrontation process. The reason lies in the synergistic effect brought about by rapid target allocation in a many-to-many confrontation situation. During the confrontation, the blue UAVs failed in the end because they only focused on pursuing and neglected to evade.

4.5. Analysis of the Impact of Termination Conditions

In this article, the capture area is set with a line of sight angle < 4° and a distance < 4 km, and the captured area is set with a line of sight angle < 20° and a distance < 4 km. To further study the impact of termination conditions on scalability, we adopted different termination conditions. To obtain the win rate, 30 Monte Carlo confrontations are conducted between the method proposed in this paper and the rule opponent. The selection of termination conditions is divided into similar conditions, simple conditions, and strict conditions.

Among them, the similar conditions are as follows: the capture area has a line of sight angle < 5° and a distance < 5 km, and the captured area has a line of sight angle < 30° and a distance < 5 km.

The simple conditions are as follows: the capture area has a line of sight angle < 60° and a distance < 8 km, and the captured area has a line of sight angle < 90° and a distance < 8 km.

The strict conditions are as follows: the capture area has a line of sight angle of less than 4° and a distance of less than 2 km, and the captured area has a line of sight angle of less than 4° and a distance of less than 2 km.

It can be seen from Table 4 that under similar conditions, the termination condition has little impact on the win rate of the method proposed in this paper. However, under simple conditions, the win rates of the red and blue sides are comparable. The reason is that the termination condition is too simple, which leads to too much randomness in meeting the termination condition, thus resulting in comparable win rates. Under harsh conditions, due to overly strict termination conditions, many scenarios ended in draws. When the scale increased, as the method proposed in this paper could produce a synergistic effect, the win rate rose to a certain extent. This also reflects the rationality of the termination conditions set in this article.

Table 4. Table of termination conditions and win rate.

	Initial Conditions	Similar Conditions	Simple Conditions	Strict Conditions
3V3	63.8%	60.8%	53.4%	25.4%
6V6	76.5%	73.5%	48.3%	32.5%
9V9	84.7%	79.4%	55.6%	38.5%
12V12	88.0%	85.6%	59.1%	41.3%

4.6. Failed Case Analysis

By analyzing the failed cases, the deficiencies of the method proposed in this paper can be obtained. We obtained a failed case in a 3V3 scenario, and the confrontation process is shown in Figure 21.

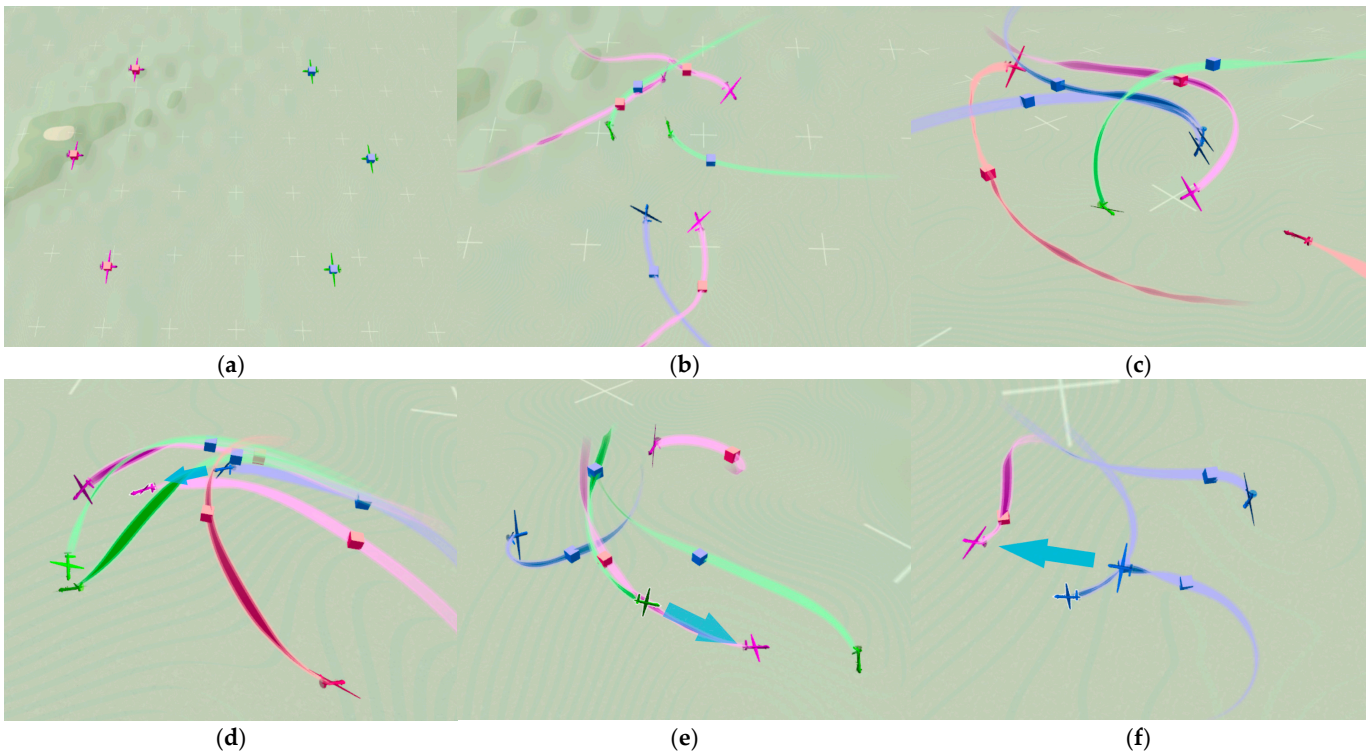


Figure 21. The game process of a failed case in a 3V3 scenario. (**a**) 0 s; (**b**) 48 s; (**c**) 119 s; (**d**) 156 s; (**e**) 166 s; (**f**) 352 s.

Figure 21 shows a failed case of a 3V3 scenario pursuit and evasion game. In Figure 21a, at the initial moment, the red and blue sides form a formation and face each other directly. In Figure 21c, a red UAV is simultaneously targeted and attacked by three blue UAVs. Then, at 156 s, this red UAV meets the termination condition. Subsequently, as the number of blue UAVs is greater than that of the red UAVs, another red UAV meets the termination condition at 166 s. Finally, at 352 s, the last red UAVs reach the termination condition one by one under the coordinated pursuit of the two blue UAVs. It can be seen from the confrontation process that the red side lost because the blue side just formed a coordinated pursuit tactic.

4.7. Ablation Experiment

To demonstrate the effectiveness of each module of the method proposed in this paper, it was verified through ablation experiments. The comparison algorithms mainly include the following:

(1) Baseline (Ours): The complete algorithm.

(2) Ablation variant 1: Ours w/o DTA, removing the dynamic target assignment and replacing it with the nearest distance assignment method.

(3) Ablation variant 2: Ours w/o TP, removing trajectory prediction.

(4) Ablation variant 3: Ours w/o SAG, removing the stable auxiliary gradient.

(5) Ablation variant 4: Ours w/o HL, removing the hierarchical reinforcement learning framework and using single-layer reinforcement learning.

The comparison results are shown in Table 5.

Table 5. Multi-scale comparison results.

	3V3		6V6		9V9		12V12	
	Win Rate	**Average OSSM**	**Win Rate**	**Average OSSM**	**Win Rate**	**Average OSSM**	**Win Rate**	**Average OSSM**
Ours (baseline)	63.8%	0.32	76.5%	0.35	84.7%	0.38	88.0%	0.34
Ours w/o DTA	56.3%	0.33	50.1%	0.36	42.6%	0.39	31.5%	0.32
Ours w/o TP	52.3%	0.31	68.5%	0.36	73.5%	0.35	76.7%	0.31
Ours w/o SAG	60.6%	0.85	72.5%	0.83	80.3%	0.81	83.5%	0.86
Ours w/o HL	51.6%	1.23	59.5%	1.18	64.6%	1.33	71.5%	1.21

By comparing the baseline and Ours w/o DTA in Table 5, in the 12v12 scenario, the win rate of Ours w/o DTA is 57.5% lower than that of the benchmark group, while in the 3v3 scenario, it is only 7.5% lower. This indicates that the role of dynamic DTA is more crucial in large-scale scenarios. The comparison between Ours w/o TP and the baseline shows that the win rate of the baseline is 9–12% higher than that of Ours w/o TP, which proves the effectiveness of trajectory prediction in improving the win rate. In the comparison between Ours w/o SAG and the baseline, OSSM was 2.7 times that of the baseline group, and the win rate decreased by 4.5–5.9%, proving that SAG ensures smoothness without sacrificing game performance. The comparison between the baseline and Ours w/o HL proves the effectiveness of hierarchical reinforcement learning in solving the coupling of agent flight control and maneuvering confrontation training and improving the win rate.

From the above simulation results, it can be seen that compared with the traditional fixed allocation and deep reinforcement learning game methods, the scalability of this framework can adapt to security requirements of different scales, and it can be quickly deployed without repeated training. Moreover, the win rate can still remain at around 80% in large-scale scenarios, meeting the protection requirements of civil airport airspace.

5. Conclusions

To address the universality of deep reinforcement learning methods in pursuit–evasion game scenarios of different scales and the stability of flight trajectories, this paper combines hierarchical reinforcement learning methods with target allocation methods and proposes a hierarchical collaborative game framework. The hierarchical collaborative game framework is divided into the target allocation layer, the maneuvering decision-making layer, and the flight control layer. To solve the problems of rapid target allocation, a dynamic target allocation layer based on a dynamic value adjustment mechanism is proposed. To improve the win rate of one-on-one pursuit–evasion game, a maneuvering decision-making method based on trajectory prediction and hierarchical reinforcement learning is proposed. To generate a smooth flight trajectory, a reinforcement learning flight control method with stable auxiliary gradients is proposed. The method proposed in this paper is effective in adversarial scenarios of any scale and has stronger interpretability compared with multi-agent reinforcement learning (MARL) methods. This explainability is verified and quantitatively supported by both the structural design and the experimental results.

Firstly, the hierarchical collaborative framework breaks down the complex multi-drone pursuit and evasion problem into three functionally independent modules: target allocation, maneuvering decision-making, and flight control. The input and output logic of each module is aligned with human tactical intuition, and the structure is transparent. Secondly, the ablation experiment (Table 5) quantified the performance contributions of each module: for instance, after removing the dynamic target allocation module, the win rate in the 12V12 scenario dropped by 57.5%, clearly indicating that it is the core of large-scale collaboration. After removing trajectory prediction, the win rate decreased by 9%

to 12%, verifying its guiding role in maneuvering decisions. Thirdly, the decision rules of the method are traceable. The dynamic target allocation builds a utility function based on the situation advantage and threat level, and the maneuverable decision-making achieves pre-tracking through trajectory prediction, all of which are consistent with human decision-making logic rather than the black-box joint learning of the MARL method. Finally, the scale-independent training of the framework (Figure 17c) essentially stems from modular disassembly, avoiding the credit allocation problem that is difficult to explain in the MARL method. These results collectively indicate that the interpretability of the proposed method is not only at the structural level but also quantitatively verified through experiments.

The method proposed in this paper can be applied to the fixed-wing UAV clusters for civil airport security. As a defensive control system, it can perform the tasks of driving away and capturing illegally invading UAVs, effectively protecting the airspace of civil airports. Future research will further explore communication and sensors, introduce dynamic leader selection mechanisms and local communication mechanisms between UAVs, and better enhance the collaborative capabilities among UAVs.

Author Contributions: Conceptualization, M.T. and D.D.; methodology, H.S. and Y.L.; validation, D.D.; formal analysis, M.T. and H.S.; writing—original draft preparation, M.T.; writing—review and editing, M.T., H.S. and D.D.; funding acquisition, H.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China (No. 62101590).

Data Availability Statement: The data that support the findings of this study are available on request from the corresponding author [Dali Ding] upon reasonable request.

DURC Statement: The current research is limited to a UAV pursuit–evasion game, which is beneficial for civil airport airspace protection and does not pose a threat to public health or national security. The authors acknowledge the dual-use potential of research involving UAV pursuit–evasion games and confirm that all necessary precautions have been taken to prevent potential misuse. As an ethical responsibility, the authors strictly adhere to the relevant national and international laws about DURC. The authors advocate for responsible deployment, ethical considerations, regulatory compliance, and transparent reporting to mitigate misuse risks and foster beneficial outcomes. All authors have read and agreed to the published version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Lyu, M.; Zhao, Y.; Huang, C.; Huang, H. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sens.* **2023**, *15*, 3266. [CrossRef]
2. Ahmed, S.; Chowdhury, M.Z.; Jang, Y.M. Energy-efficient UAV relaying communications to serve ground nodes. *IEEE Commun. Lett.* **2020**, *24*, 849–852. [CrossRef]
3. Khan, A.; Gupta, S.; Gupta, S.K. Emerging UAV technology for disaster detection, mitigation, response, and preparedness. *J. Field Robot.* **2022**, *39*, 905–955. [CrossRef]
4. Kang, H.; Joung, J.; Kim, J.; Kang, J.; Cho, Y.S. Protect your sky: A survey of counter unmanned aerial vehicle systems. *IEEE Access* **2020**, *8*, 168671–168710. [CrossRef]
5. Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A. Mastering the game of go without human knowledge. *Nature* **2017**, *550*, 354–359. [CrossRef]
6. Berner, C.; Brockman, G.; Chan, B.; Cheung, V.; Dębiak, P.; Dennison, C.; Farhi, D.; Fischer, Q.; Hashme, S.; Hesse, C. Dota 2 with large scale deep reinforcement learning. *arXiv* **2019**, arXiv:1912.06680. [CrossRef]
7. Vinyals, O.; Babuschkin, I.; Czarnecki, W.M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D.H.; Powell, R.; Ewalds, T.; Georgiev, P. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **2019**, *575*, 350–354. [CrossRef] [PubMed]
8. Li, J.; Koyamada, S.; Ye, Q.; Liu, G.; Wang, C.; Yang, R.; Zhao, L.; Qin, T.; Liu, T.-Y.; Hon, H.-W. Suphx: Mastering mahjong with deep reinforcement learning. *arXiv* **2020**, arXiv:2003.13590. [CrossRef]

9. Perolat, J.; De Vylder, B.; Hennes, D.; Tarassov, E.; Strub, F.; de Boer, V.; Muller, P.; Connor, J.T.; Burch, N.; Anthony, T. Mastering the game of Stratego with model-free multiagent reinforcement learning. *Science* **2022**, *378*, 990–996. [CrossRef]
10. Crumpacker, J.B.; Robbins, M.J.; Jenkins, P.R. An approximate dynamic programming approach for solving an air combat maneuvering problem. *Expert Syst. Appl.* **2022**, *203*, 117448. [CrossRef]
11. Zhang, J.; Dinghan, W.; Qiming, Y.; Zhuoyong, S.; Longmeng, J.; Guoqing, S.; Yong, W. Loyal wingman task execution for future aerial combat: A hierarchical prior-based reinforcement learning approach. *Chin. J. Aeronaut.* **2024**, *37*, 462–481. [CrossRef]
12. Yang, Q.; Zhu, Y.; Zhang, J.; Qiao, S.; Liu, J. UAV air combat autonomous maneuver decision based on DDPG algorithm. In Proceedings of the 2019 IEEE 15th International Conference on Control And Automation (ICCA), Edinburgh, UK, 16–19 July 2019; pp. 37–42.
13. Gao, X.; Zhang, Y.; Wang, B.; Leng, Z.; Hou, Z. The Optimal Strategies of Maneuver Decision in Air Combat of UCAV Based on the Improved TD3 Algorithm. *Drones* **2024**, *8*, 501. [CrossRef]
14. Li, B.; Huang, J.; Bai, S.; Gan, Z.; Liang, S.; Evgeny, N.; Yao, S. Autonomous air combat decision-making of UAV based on parallel self-play reinforcement learning. *CAAI Trans. Intell. Technol.* **2023**, *8*, 64–81. [CrossRef]
15. Freitas, A.; Rabbath, C.A.; Williams, C.; Lechevin, N.; Givigi, S. Multi-Unmanned Aerial Vehicles Cooperative Tactics: A Survey. *IEEE Access* **2025**, *13*, 119897–119921. [CrossRef]
16. Pang, J.; He, J.; Mohamed, N.M.A.A.; Lin, C.; Zhang, Z.; Hao, X. A hierarchical reinforcement learning framework for multi-UAV combat using leader–follower strategy. *Knowl.-Based Syst.* **2025**, *316*, 113387. [CrossRef]
17. Qu, P.; He, C.; Wu, X.; Wang, E.; Xu, S.; Liu, H.; Sun, X. Double mixing networks based monotonic value function decomposition algorithm for swarm intelligence in UAVs. *Auton. Agents Multi-Agent Syst.* **2025**, *39*, 16. [CrossRef]
18. Lei, L.; Wu, C.; Chen, H. Dynamic Decoupling-Driven Cooperative Pursuit for Multi-UAV Systems: A Multi-Agent Reinforcement Learning Policy Optimization Approach. *Comput. Mater. Contin.* **2025**, *85*, 1339–1363. [CrossRef]
19. Liang, X.; Wang, H.; Luo, H. Collaborative Pursuit-Evasion Strategy of UAV/UGV Heterogeneous System in Complex Three-Dimensional Polygonal Environment. *Complexity* **2020**, *2020*, 7498740. [CrossRef]
20. Shahar, F.S.; Sultan, M.T.H.; Nowakowski, M.; Łukaszewicz, A. UGV-UAV Integration Advancements for Coordinated Missions: A Review. *J. Intell. Robot. Syst.* **2025**, *111*, 69. [CrossRef]
21. Pope, A.P.; Ide, J.S.; Mićović, D.; Diaz, H.; Twedt, J.C.; Alcedo, K.; Walker, T.T.; Rosenbluth, D.; Ritholtz, L.; Javorsek, D. Hierarchical Reinforcement Learning for Air Combat at DARPA's AlphaDogfight Trials. *IEEE Trans. Artif. Intell.* **2023**, *4*, 1371–1385. [CrossRef]
22. Piao, H.; Han, Y.; He, S.; Yu, C.; Fan, S.; Hou, Y.; Bai, C.; Mo, L. Spatiotemporal Relationship Cognitive Learning for Multirobot Air Combat. *IEEE Trans. Cogn. Dev. Syst.* **2023**, *15*, 2254–2268. [CrossRef]
23. Qian, C.; Zhang, X.; Li, L.; Zhao, M.; Fang, Y. H3E: Learning air combat with a three-level hierarchical framework embedding expert knowledge. *Expert Syst. Appl.* **2024**, *245*, 123084. [CrossRef]
24. Kong, W.R.; Zhou, D.Y.; Du, Y.J.; Zhou, Y.; Zhao, Y.Y. Hierarchical multi-agent reinforcement learning for multi-aircraft close-range air combat. *IET Control Theory Appl.* **2023**, *17*, 1840–1862. [CrossRef]
25. Xu, X.; Wang, Y.; Guo, X.; Huang, K.; Zhang, X. Multi-UAV air combat cooperative game based on virtual opponent and value attention decomposition policy gradient. *Expert Syst. Appl.* **2025**, *267*, 126069. [CrossRef]
26. Yu, C.; Velu, A.; Vinitsky, E.; Gao, J.; Wang, Y.; Bayen, A.; Wu, Y. The surprising effectiveness of ppo in cooperative multi-agent games. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24611–24624.
27. Rashid, T.; Samvelyan, M.; De Witt, C.S.; Farquhar, G.; Foerster, J.; Whiteson, S. Monotonic value function factorisation for deep multi-agent reinforcement learning. *J. Mach. Learn. Res.* **2020**, *21*, 1–51.
28. Zhang, Y.; Ding, M.; Zhang, J.; Yang, Q.; Shi, G.; Lu, M.; Jiang, F. Multi-UAV pursuit-evasion gaming based on PSO-M3DDPG schemes. *Complex Intell. Syst.* **2024**, *10*, 6867–6883. [CrossRef]
29. Zhang, Y.; Zhu, Y.; Wang, J. Gaussian-enhanced reinforcement learning for scalable evasion strategies in multi-agent pursuit-evasion games. *Neurocomputing* **2025**, *652*, 131080. [CrossRef]
30. Wang, B.; Gao, X.; Xie, T. An evolutionary multi-agent reinforcement learning algorithm for multi-UAV air combat. *Knowl.-Based Syst.* **2024**, *299*, 112000. [CrossRef]
31. Abadi, A.S.; Soh, L.-K. Challenges in Credit Assignment for Multi-Agent Reinforcement Learning in Open Agent Systems. *arXiv* **2025**, arXiv:2510.27659. [CrossRef]
32. Zhang, S.; Zhou, J.; Tian, D.; Sheng, Z.; Duan, X.; Leung, V.C. Robust cooperative communication optimization for multi-UAV-aided vehicular networks. *IEEE Wirel. Commun. Lett.* **2020**, *10*, 780–784. [CrossRef]
33. Lv, Z.; Xiao, L.; Du, Y.; Niu, G.; Xing, C.; Xu, W. Multi-agent reinforcement learning based UAV swarm communications against jamming. *IEEE Trans. Wirel. Commun.* **2023**, *22*, 9063–9075. [CrossRef]
34. De Marco, A.; D'Onza, P.M.; Manfredi, S. A deep reinforcement learning control approach for high-performance aircraft. *Nonlinear Dyn.* **2023**, *111*, 17037–17077. [CrossRef]

35. Littman, M.L. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994*; Elsevier: Amsterdam, The Netherlands, 1994; pp. 157–163.
36. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347. [CrossRef]
37. Xu, X.; Zuo, L.; Li, X.; Qian, L.; Ren, J.; Sun, Z. A Reinforcement Learning Approach to Autonomous Decision Making of Intelligent Vehicles on Highways. *IEEE Trans. Syst. Man Cybern.-Syst.* **2020**, *50*, 3884–3897. [CrossRef]
38. Wang, P.; Chan, C.-Y.; de la Fortelle, A. A Reinforcement Learning Based Approach for Automated Lane Change Maneuvers. In Proceedings of the IEEE Intelligent Vehicles Symposium (IV), Changshu, China, 26–30 June 2018; pp. 1379–1384.
39. Zhu, M.; Wang, Y.; Pu, Z.; Hu, J.; Wang, X.; Ke, R. Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving. *Transp. Res. Part C-Emerg. Technol.* **2020**, *117*, 102662. [CrossRef]
40. Ren, Z.; Zhang, D.; Tang, S.; Xiong, W.; Yang, S.-h. Cooperative maneuver decision making for multi-UAV air combat based on incomplete information dynamic game. *Def. Technol.* **2023**, *27*, 308–317. [CrossRef]
41. Chae, H.-J.; Choi, H.-L. Tactics Games for Multiple UCAVs Within-Visual-Range Air Combat. In Proceedings of the 2018 AIAA Information Systems-AIAA Infotech @ Aerospace, Kissimmee, FL, USA, 8–12 January 2018.
42. Li, F.; Yin, M.; Wang, T.; Huang, T.; Yang, C.; Gui, W. Distributed pursuit-evasion game of limited perception USV swarm based on multiagent proximal policy optimization. *IEEE Trans. Syst. Man Cybern. Syst.* **2024**, *54*, 6435–6446. [CrossRef]
43. Wang, H.; Wang, J. Enhancing multi-UAV air combat decision making via hierarchical reinforcement learning. *Sci. Rep.* **2024**, *14*, 4458. [CrossRef] [PubMed]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.